Java 基础面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

你使用过哪些 JDK 提供的工具？

JDK 自带的工具其实是日常排查问题的利器，很多场景下压根不需要额外依赖第三方工具。

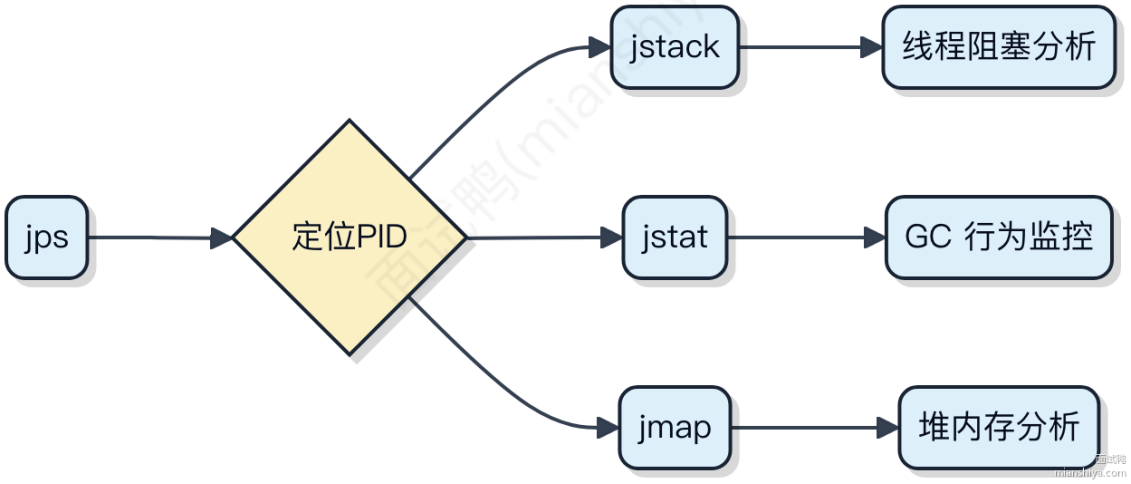
jps 相当于 Java 版的 ps，能快速列出当前机器上所有正在运行的 Java 进程 PID。排查多实例部署时特别有用，几秒钟就能定位到目标进程。

jstat 是看 JVM 运行状态最轻量的方式，尤其是监控 GC 频率和堆内存变化。比如想确认是不是频繁 Full GC，用 `jstat -gcutil <pid> 1000` 每秒打一次数据，趋势一目了然。

jstack 对应线程分析，定位死锁或高 CPU 很直接。拿到进程 PID 后执行 `jstack <pid>` ，输出的线程栈里如果看到 WAITING 状态集中在某个锁对象，基本就能锁定问题点。线上服务响应卡顿，第一反应就是抓 thread dump 看有没有线程堆积。

jmap 可以导出堆内存快照，配合 jhat 或 MAT 分析内存泄漏。不过要注意的是， `jmap -dump` 在堆大的时候会触发长时间 STW，生产环境得谨慎操作。

其实从 JDK 8 开始，**Arthas** 虽然是阿里开源的，但已经成了事实标准。它把上面这些命令都整合了，还能动态 trace 方法调用、watch 参数返回值，排查线上问题效率提升非常明显。



什么是 Selector？

Selector 是 Java NIO 里用来做事件监听的核心组件，一个线程通过它就能同时监控多个通道的 I/O 状态，比如有没有数据可读、能不能写入。

你想想，传统 IO 一个连接就得占一个线程，1000 个连接就得分 1000 个线程去处理，系统压根扛不住。而用了 Selector，一个线程轮询多个 Channel 的就绪事件，资源利用率直接拉满，这就是为什么 Netty、Dubbo 这些高性能中间件底层都靠它撑着。

每个 Channel 注册到 Selector 上时得绑定一个 SelectionKey，表示“我对哪些事件感兴趣”，比如 OP_READ、OP_WRITE。调用 select() 方法后，线程会阻塞直到有至少一个通道就绪，然后返回就绪的 key 集合，挨个处理就行。

代码上大概是这样：

```
Selector selector = Selector.open();
channel.configureBlocking(false);
channel.register(selector, SelectionKey.OP_READ);

while (true) {
    int readyChannels = selector.select(); // 阻塞等待就绪事件
    if (readyChannels == 0) continue;

    Set<SelectionKey> keys = selector.selectedKeys();
    for (SelectionKey key : keys) {
        if (key.isReadable()) { /* 处理读 */ }
        keys.remove(key);
    }
}
```

注意非阻塞模式是前提，阻塞 IO 调了 select() 也没意义。另外 select() 返回的是就绪的通道数，不是所有注册的。

整个模型叫多路复用，Linux 上实际是 epoll 在背后干活。

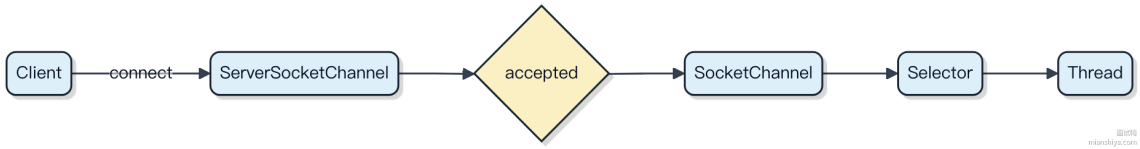
什么是 Channel?

Channel 是 Java NIO 的核心组件，可以看作是数据传输的通道，负责从缓冲区读写数据。它和传统的 IO 流不同，Channel 是双向的，既能读也能写，而流一般是单向的。

1）常见的 Channel 实现有 FileChannel、SocketChannel、ServerSocketChannel 和 DatagramChannel。比如 Netty 就基于 NioSocketChannel 做网络通信，用统一的抽象屏蔽底层差异。

2）Channel 本身不直接操作数据，数据总是流向 Buffer。读数据时，数据从 Channel 进入 Buffer；写数据时，数据从 Buffer 写入 Channel。这种设计让数据处理更灵活，也便于零拷贝技术的应用。

3）在高并发场景下，Channel 配合 Selector 实现多路复用。一个线程就能监控多个 Channel 的事件，比如连接就绪、读就绪，避免为每个连接开线程，扛住几万并发连接很常见。



使用时要注意，Channel 必须配置为非阻塞模式才能注册到 Selector，否则会抛异常。另外，关闭 Channel 会释放对应的文件描述符，这个资源很宝贵，Linux 下一般默认最多 1024 个，用完就得调优。

如果一个线程在 Java 中被两次调用 start() 方法，会发生什么？

调用 `start()` 方法的本质是让 JVM 将线程加入调度队列，真正触发 `run()` 的执行。线程的状态机决定了它只能从 **NEW** 状态被启动一次。

1）第一次调用 `start()` ，线程状态从 NEW 变为 RUNNABLE，等待 CPU 调度执行 `run()` 方法。

2）第二次再调用 `start()` ，JVM 会检查线程当前状态，发现已经不是 NEW，直接抛出 `IllegalThreadStateException` 。

这个异常是运行时异常，不强制捕获，但一旦发生程序就会中断。常见于误把线程对象当工具复用，比如在循环里反复启动同一个线程。

```
Thread t = new Thread(() -> System.out.println("hello"));
t.start();
t.start(); // 这里炸了
```

正确做法是每次都需要新线程，就得 new 一个新实例，或者用线程池管理生命周期。像 ThreadPoolExecutor 这种，你 submit 任务，它背后复用 worker 线程，不会出现重复启动的问题。

根本原因在于 Thread 类的设计就是“一次性”的，run 方法执行完，线程生命周期就结束了，不能回退到 NEW 状态重来。

Java 的 Optional 类是什么？它有什么用？

Optional 其实是 Java 8 引入的一个容器类，用来包装可能为 null 的值。它的核心意图不是消灭 null，而是让开发者明确表达“这里可能没有值”，从而减少空指针异常。

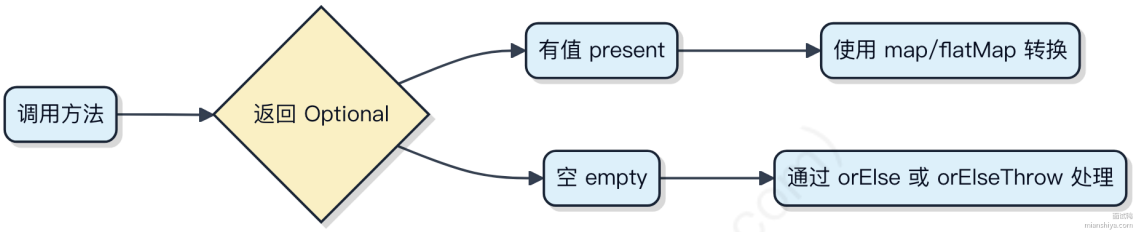
用 Optional 后，方法返回类型会直接告诉调用方：这个结果**可能不存在**。比如 `Optional<String>` 比 `String` 更清晰地传达语义，避免隐式 null 带来的误解。

常见用法有几种：1）构建 Optional：`Optional.of(value)` 用于非 null 值，`Optional.ofNullable(value)` 可处理 null。2）取值：`orElse(default)` 提供默认值，`orElseThrow()` 在无值时抛异常。3）链式操作：`map()` 对值进行转换，`filter()` 进行条件过滤，都不用担心 NPE。

```
Optional<String> name = Optional.ofNullable(getName());
String result = name.map(String::toUpperCase)
                    .orElse("UNKNOWN");
```

但别滥用。**Optional** 不该用在集合元素里，也不该作为类字段——它不是序列化的友好选择。像 Guava 早期就有类似设计，但 Java 8 的标准库支持让它成了主流做法。

它更适合用在方法返回值上，尤其是工具类或查找方法，比如 `findUserById()` 返回 `Optional<User>` ，调用方自然知道要处理“找不到”的情况。



Java 的 I/O 流是什么？

Java 的 I/O 流本质上是数据传输的抽象模型，把数据的输入输出看作“流动”的字节或字符。它不关心数据源具体是文件、网络还是内存，只关注“从哪来、到哪去”这个流动过程。

输入流负责读，输出流负责写。字节流处理 8 位的原始二进制数据，比如图片、视频，用 `InputStream` 和 `OutputStream` 这两个抽象基类。字符流则面向文本，自动处理编码转换，基于 `Reader` 和 `Writer`，适合读写字符串。

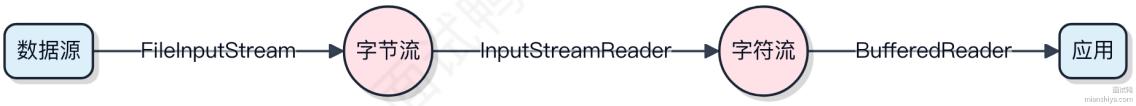
实际开发中不会直接用这些顶层抽象类，而是组合具体实现。比如要高效读文件，会用 `BufferedInputStream` 包一层 `FileInputStream`，缓冲机制能减少系统调用次数，提升性能。网络通信里常见的 `ObjectOutputStream` 就是用来序列化对象发到远端。

1）字节流处理二进制，比如 `FileInputStream` 读图片
2）字符流处理文本，避免乱码，比如 `InputStreamReader` 转换编码
3）装饰器模式很关键，`Buffered`、`Data`、`Object` 这些都是增强功能

举个例子，读一个 UTF-8 文本文件：

```
try (var br = new BufferedReader(
        new InputStreamReader(
          new FileInputStream("data.txt"), "UTF-8"))) {
    br.lines().forEach(System.out::println);
}
```

现在更推荐用 NIO 的 `Files.newBufferedReader(Paths.get("data.txt"))`，一行搞定，底层自动处理编码和缓冲。



Java 中的基本数据类型有哪些？

Java 的基本数据类型一共 8 种，是构建程序的最基础单元。它们直接存储值，不涉及对象引用，性能高，用起来也简单。

整数类型有 4 种：
1） `byte`：1 字节，范围 -128 到 127，适合节省内存的场景，比如数组里存大量小数字。
2） `short`：2 字节，范围 -32768 到 32767，用得不多，偶尔在文件格式或网络协议里见。

3） `int` ：4 字节，最常用，一般循环计数、数学计算都用它。
4） `long` ：8 字节，大数才用，比如时间戳 `System.currentTimeMillis()` 返回的就是 long。

浮点类型 2 种：
5） `float` ：4 字节，单精度，声明时要加 f，比如 `3.14f` ，科学计算或图形处理可能用到。
6） `double` ：8 字节，双精度，日常小数计算的主力，比如 `3.14` 默认就是 double。

字符类型 1 种：
7） `char` ：2 字节，存 Unicode 字符，比如 `'a'` 或 `'汉'` ，本质是无符号整数，可以做加减。

布尔类型 1 种：
8） `boolean` ：真假值，只有 `true` 和 `false` ，控制逻辑分支，JVM 内部其实用 1 字节实现，虽然理论上只需要 1 位。

这些类型对应各自的包装类，比如 `Integer` 、 `Boolean` ，用在集合或者需要 null 的场景。但**自动装箱拆箱**有性能损耗，高频场景要小心。

什么是 Java 中的自动装箱和拆箱？

Java 里的自动装箱和拆箱，其实就是编译器在基本类型和对应的包装类之间**自动转换**的语法糖。你写代码的时候不用手动调 `Integer.valueOf()` 或 `.intValue()` ，但底层其实都给你处理了。

1）自动装箱是指把基本类型转成包装对象，比如 `Integer i = 100;` 这行代码，编译后其实是 `Integer i = Integer.valueOf(100);` 。这里有个关键点， `valueOf` 在 -128 到 127 范围内会用缓存对象，所以这个区间内的 Integer 比较可以用 ==，超出就得用 equals。

2）拆箱反过来，是从包装类取基本值，像 `int j = new Integer(100);` 会被编译成调用 `intValue()` 。但如果对象是 null，拆箱时就会抛出 `NullPointerException` ，这种空指针问题在集合操作里特别容易踩坑。

```
List<Integer> list = new ArrayList<>();
list.add(100); // 装箱
int value = list.get(0); // 拆箱，如果 get 出 null 就 NPE
```

这种机制用起来方便，但在高频循环或性能敏感场景要小心，频繁创建包装对象会增加 GC 压力。像 Kafka、Netty 这些高性能中间件里，能用 primitive 就不用 wrapper，就是这个道理。

Java 中 for 循环与 foreach 循环的区别是什么？

Java 里的 for 和 foreach 看似都能遍历，但底层机制和适用场景其实有挺大差别。

1）传统 for 循环靠索引控制，你得自己管理下标，适合需要访问索引的场景。比如你想处理数组中偶数位的元素，或者反向遍历，这时候用 for 更直接。

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

2）foreach（增强 for）本质是 **迭代器** 的语法糖，编译后会转成 `Iterator` 的 `hasNext()` 和 `next()` 调用。它屏蔽了索引细节，代码更简洁，也避免越界错误。但正因为没有索引，你没法在遍历过程中修改集合（比如 remove 不通过迭代器会抛 ConcurrentModificationException）。

```
for (String s : list) {
    System.out.println(s);
}
```

3）性能上，数组类型两者基本没差，JVM 会优化。但对 `ArrayList` 这种实现了 RandomAccess 的集合，for 通过 get(i) 访问很快。而 `LinkedList` 用 foreach 更好，因为每次 get(i) 都要从头遍历，$O(n^2)$ 的代价，压根不推荐。

4）foreach 不能用于需要并发修改的场景，也不能做条件跳步（比如 i += 2）。遇到这些情况，老老实实用传统 for 或显式迭代器。

总的来说，能用 foreach 就用，代码干净安全。需要索引或复杂控制逻辑时，再切回 for。

你使用过 Java 的反射机制吗？如何应用反射？

Java 反射这东西，平时写业务代码可能不常碰，但框架里到处都是它的影子。你用 Spring 的时候，那个 `@Autowired` 注入、`@RequestMapping` 映射，底层全靠反射搞定。

1）运行时动态操作类是反射的核心能力。比如你有个字符串 `"com.example.User"`，想在程序跑起来之后创建这个类的实例，常规的 new 是做不到的，因为编译时根本不知道具体类型。这时候 Class.forName 加 newInstance（或者现在的 getDeclaredConstructor().newInstance()）就能派上用场。

2）访问私有成员也是常见用途。单元测试里有些 private 方法要测，或者像某些工具类需要绕过访问限制，通过 setAccessible(true) 就能强行读写。不过这招别乱用，破坏了封装性，维护起来头疼。

3）典型的场景就是 ORM 框架。像 MyBatis 处理结果集映射时，查出一行数据，它得知道怎么塞进 User 对象的字段里。通过反射拿到 ResultMap 定义的属性名，再找对应的 setter 或字段直接赋值，完全不用提前写死转换逻辑。

```
Class<?> clazz = Class.forName("com.example.User");
Object obj = clazz.getDeclaredConstructor().newInstance();
Field field = clazz.getDeclaredField("name");
field.setAccessible(true);
field.set(obj, "John");
```

性能方面，反射比直接调用慢个几倍到几十倍，关键路径上频繁使用会拖垮系统。所以像 JSON 解析库（如 Jackson）会在首次反射后缓存 Method/Field 引用，后续复用提升效率。

要不要用？框架作者躲不开，业务开发尽量少碰。真要用，记得做缓存，别让反射成为瓶颈。

什么是 Java 中的继承机制？

Java 里的继承，说白了就是子类可以拿父类的属性和方法来用，不用重复写。一个类只能继承一个父类，这是单继承，靠的是 `extends` 关键字。

1）子类会自动拥有父类的非私有字段和方法。比如你写个 `Animal` 类有 `eat()` 方法，`Dog extends Animal` 就可以直接调用 `eat()`。

2）构造过程是先跑父类构造器，再跑子类的。如果你没手动调 `super()`，编译器会自动插一句 `super()` 在子类构造器第一行。

3）方法重写（Override）是重点。子类可以改写父类的方法逻辑，但签名得一样。加个 `@Override` 注解能防止你写错签名，也方便别人看。

```java
class Animal {
    void sound() { System.out.println("叫了一声"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("汪汪"); }
}
```

多态也是基于继承来的。你用 `Animal a = new Dog()` 这种写法，调 `a.sound()` 实际执行的是 `Dog` 的版本，这就是运行时动态绑定。

不过别滥用继承。父类一改，子类可能就出问题。一般来说，**is-a** 关系才考虑继承，比如 `Dog is an Animal` 。要是只是想复用代码，优先用组合，不然后期搞不定。

Java 中的访问修饰符有哪些？

Java 里的访问修饰符主要就四个：`private` 、`default` （也叫包访问权限）、`protected` 和 `public` 。它们控制的是类、方法、变量这些成员能被访问的范围。

1） `private` 最严格，只能在定义它的那个类内部访问，别的类哪怕同一个包都不行。

2） `default` 是不写任何修饰符时的默认行为，同一个包内的类可以访问。跨包就不行，哪怕继承也不管用。

3） `protected` 比 default 宽一点，同一个包里的类能访问，**不同包的子类**也能访问，这是它和 default 的关键区别。

4） `public` 最开放，谁都能访问，不管是不是同一个包，也不管有没有继承关系。

举个例子，你在写一个工具类的时候，核心算法可能用 private 封装起来，只暴露 public 的调用方法；而在继承体系里，父类想让子类用某个方法但又不想对外公开，就会用 protected。

修饰符	同一类	同一包	不同包子类	不同包非子类
private	✅	❌	❌	❌
default	✅	✅	❌	❌
protected	✅	✅	✅	❌
public	✅	✅	✅	✅

这个表格记熟了，基本就不会搞混了。实际开发中，优先缩小访问范围，能用 private 就不用 public，这是封装的基本原则。

Java 中静态方法和实例方法的区别是什么？

静态方法属于类本身，直接通过类名调用，不依赖任何实例。实例方法则必须在对象创建后才能调用，它能访问当前实例的成员变量。

1）内存分配时机不同。静态方法随着类加载就存在了，实例方法得等 new 出对象后才可使用。比如 `Math.max()` 是静态的，不用 new Math 对象就能用。

2）访问权限有区别。静态方法只能直接访问静态成员，不能用 this 或 super，也不能访问实例变量。实例方法啥都能访问。写工具类时常用 static，像 `Collections.sort(list)` 。

3）多态性表现不一样。实例方法支持重写，运行时动态绑定，父类引用指向子类对象能调用子类实现。静态方法没有这回事，它是编译期绑定的，子类定义同名静态方法只是隐藏父类的，不会发生**动态分派**。

```
class Parent {
    static void say() { System.out.println("Parent static"); }
    void speak() { System.out.println("Parent instance"); }
}

class Child extends Parent {
    static void say() { System.out.println("Child static"); } // 隐藏，非重写
    void speak() { System.out.println("Child instance"); }   // 重写
}
```

调用 `new Child().speak()` 输出 "Child instance"，而 `Parent.say()` 或 `Child.say()` 只看左边声明类型，和对象无关。

一般工具方法、工厂方法用静态，业务行为、需要状态的操作用实例。搞不定选型时问自己：这个方法要不要依赖对象的状态？要就上实例方法。

JDK 和 JRE 有什么区别？

JDK 是给开发者用的，它里面不仅包含了写 Java 代码需要的编译器、调试工具，还打包了 JRE。你可以把它看作一个完整的开发套件。

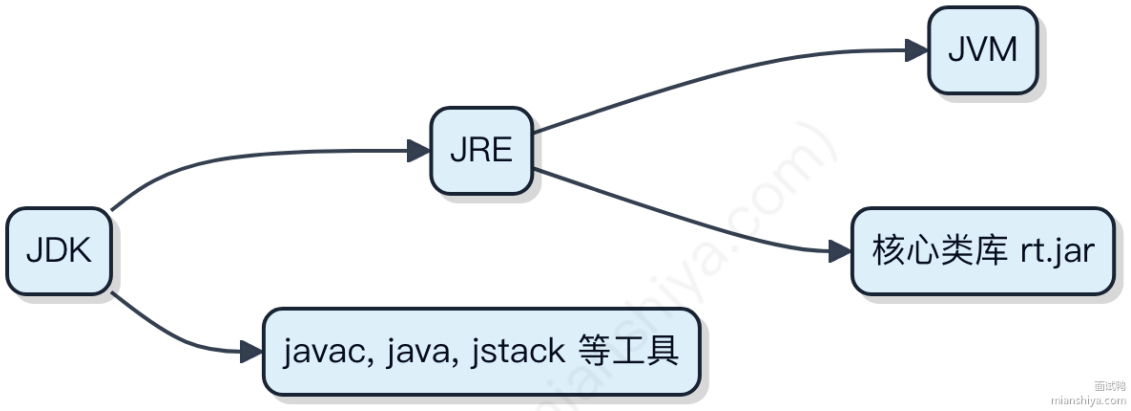
JRE 则是运行 Java 程序的基础环境，它包含 JVM 和运行时需要的核心类库。比如你只是想跑一个别人写好的 .jar 文件，装 JRE 就够了，不需要编译功能。

所以关键在于用途不同：开发选 JDK，纯运行选 JRE。

打个比方，JDK 像是整套厨房设备，有刀具、灶台、调料（编译、调试、运行全都有），而 JRE 只是一个微波炉，负责把做好的饭菜热一下（只负责运行）。

现在主流的 JDK 发行版，像 OpenJDK、Oracle JDK 或者国内常用的 Alibaba Dragonwell，其实都自带了 JRE，安装后可以直接编译也能直接运行程序。

1）JDK 包含 JRE，JRE 包含 JVM
2）没有 JDK 也能运行 Java 程序，只要有 JRE
3）javac 命令在 JRE 中不可用，因为它属于编译工具，只存在于 JDK



面试如果问到，记住一句话就行：**JDK 是开发环境，JRE 是运行环境**。

Java 中 wait() 和 sleep() 的区别？

`wait()` 和 `sleep()` 看起来都是让线程“停下来”，但它们的使用场景和底层机制完全不同。

1） `wait()` 是 `Object` 的方法，调用后线程会释放持有的**监视器锁（monitor lock）**，进入等待队列，直到其他线程执行 `notify()` 或 `notifyAll()` 才能被唤醒。它必须在 `synchronized` 块或方法中使用，否则会抛出 `IllegalMonitorStateException` 。

```
synchronized (obj) {
    obj.wait(); // 释放锁，进入等待
}
```

2） `sleep()` 是 `Thread` 类的静态方法，它只是让当前线程暂停指定时间，**不释放任何锁**。时间一到，线程进入就绪状态等待 CPU 调度。

```
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
```

关键区别在于**锁的释放与协作机制**。 `wait()` 是线程间通信的一部分，常用于生产者-消费者模式，配合 `notify()` 实现线程协作。而 `sleep()` 更像是“我先歇会儿”，跟同步无关。

还有一点， `wait()` 可以被外部唤醒， `sleep()` 虽然也能被中断，但需要捕获 `InterruptedException` ，并且不会自动重新获取锁。

所以别搞混了，想做线程协调，用 `wait()/notify()` ；只想暂停一下，用 `sleep()` 就行。

PO、VO、BO、DTO、DAO、POJO 有什么区别？

这几个对象在分层架构里各司其职，搞清楚它们的职责边界，代码才不会乱成一锅粥。

PO 是跟数据库表直接对应的实体类，字段和表列一一对应，一般用于 ORM 框架比如 MyBatis 或 JPA。它待在持久层，不往外传。

DTO 是用来传输数据的，跨服务或跨层传递时用。比如 Controller 和远程接口之间，字段可能比 PO 少，也可能组合了多个表的数据，目的就是减少网络传输量。

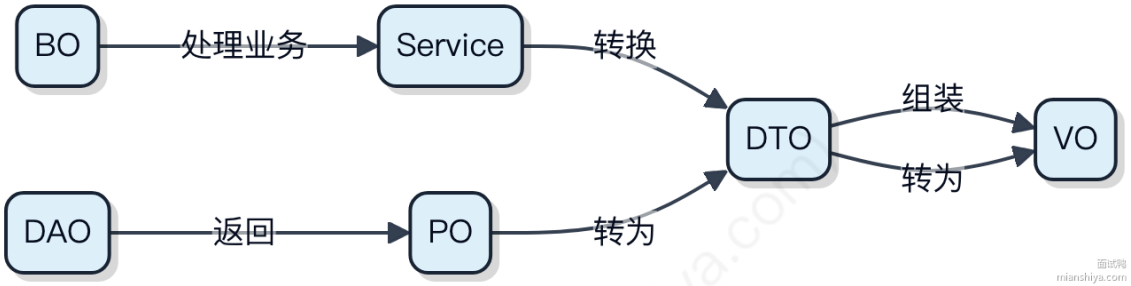
VO 一般是给前端展示用的，可能聚合了用户信息、订单状态、商品详情等，结构完全为页面定制。比如一个订单详情页要显示用户名、地址、物流进度，VO 就把这些都打包好。

BO 承担业务逻辑，可能包含一些计算方法或流程控制。比如 OrderBO 可能有 `calculateDiscount()` 或 `isOverdue()` 这种带行为的对象，通常在 service 内部流转。

DAO 是数据访问对象，专门负责操作数据库，比如 UserDao 定义了 `insertUser()` 、 `findByPhone()` 等方法。它是个接口或类，干的是和 DB 交互的脏活累活。

POJO 是最普通的 Java 对象，不依赖任何框架接口，上面这些对象本质上都是 POJO，只要没继承特殊类、没实现框架接口，就是 POJO。

1）PO → 数据库映射
2）DTO → 跨层数据搬运
3）VO → 前端视图组装
4）BO → 业务逻辑承载
5）DAO → 操作数据库的方法集合
6）POJO → 所有普通 Java 对象的统称



你认为 Java 的优势是什么？

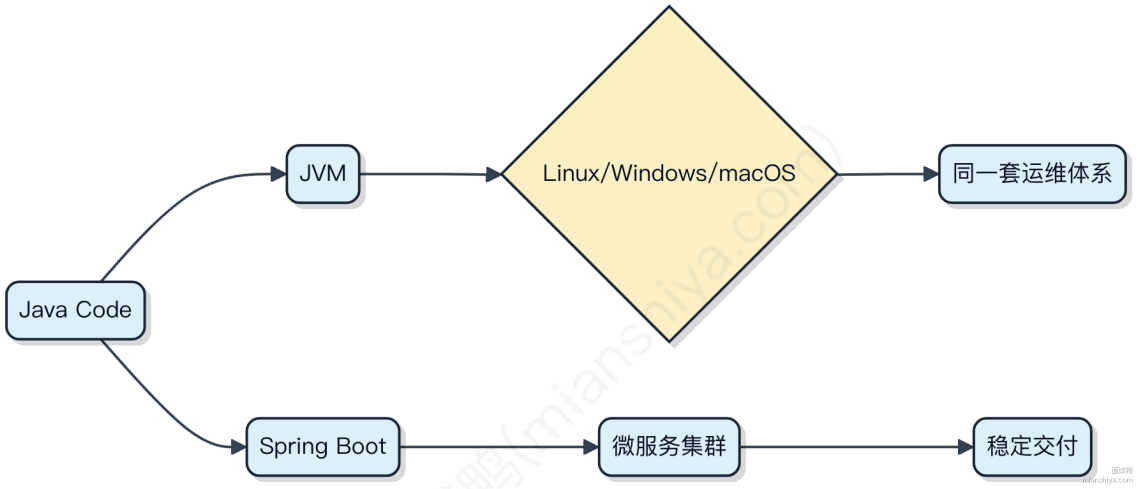
Java 的优势其实体现在工程落地的稳定性上，尤其适合大型复杂系统。

首先，**JVM** 是它的核心护城河。一次编写，到处运行不是口号，是实打实的生产力保障。你写的服务在开发机跑得好好的，扔到线上大概率还是稳的，不用操心底层操作系统差异带来的诡异问题。

类库生态也成熟得离谱。从 Spring 全家桶做 Web 服务，到 Netty 写高性能通信，再到 Kafka、Flink 这些大数据组件，清一色 Java 栈。公司里搞个微服务，Spring Boot 加几个 starter 就能快速搭起来，省了多少脏活累活。

1）内存管理自动搞定，GC 虽然偶尔会停顿，但 G1、ZGC 这些新收集器已经能把暂停压到毫秒级甚至更低
2）并发编程有完整的工具链，ConcurrentHashMap、CompletableFuture、ForkJoinPool 都是实战利器
3）语言本身虽不算炫，但泛型、注解、Lambda 这些该有的都有，代码可读性和维护性拿捏住了

企业级应用里，Java 拿来就能扛住高并发场景。像阿里早期的交易系统、银行的核心账务，都是靠 Java 堆出来的。不是它完美，而是整个技术闭环太完整，从诊断（jstack、jmap）、监控（Micrometer）、部署到调优，工具链一套接一套。



Java21 有哪些新特性？

Java 21 是一个长期支持（LTS）版本，带来了不少实用的新特性，主要集中在语法简化、性能提升和底层机制增强。

1）虚拟线程是最大亮点。它属于 Project Loom 的成果，把并发编程从“每个请求一个线程”切换到“成千上万协程共享少量 OS 线程”。以前用 Tomcat 线程池扛 1 万并发都吃力，现在用虚拟线程轻松应对。写法也简单，几乎不用改代码。

```
Thread.startVirtualThread(() -> System.out.println("运行在虚拟线程"));
```

2）结构化并发处于孵化阶段，配合虚拟线程使用。它让多个子任务的生命周期统一管理，异常传递更清晰，适合批处理或分片查询场景。

3）记录模式（Record Patterns）和数组解构是模式匹配的进一步落地。以前要写一堆 if-else instanceof 判断再拆字段，现在可以直接匹配数据结构。

```
if (obj instanceof Point(int x, int y)) {
    System.out.println(x + ", " + y);
}
```

4）字符串模板（ `STR` ）是预览功能，取代繁琐的 String.format 或 StringBuilder 拼接。写 SQL 或 JSON 组装时特别顺手。

```
String name = "Alice";
System.out.println(STR."Hello \{name}"); // Hello Alice
```

这些特性里，**虚拟线程**对系统吞吐量影响最大。像 WebFlux 这类响应式框架的部分压力其实来自回调复杂性，而虚拟线程用同步代码就能写出高并发服务，直接降低心智负担。

要不要升级？如果你的应用是典型的 I/O 密集型，比如网关、微服务接口层，升级后可能不改代码就提升 3-5 倍 QPS。但计算密集型服务收益就不明显。

Java 方法重载和方法重写之间的区别是什么？

方法重载和重写看起来都是“同名不同义”，但它们的使用场景和底层逻辑完全不同。

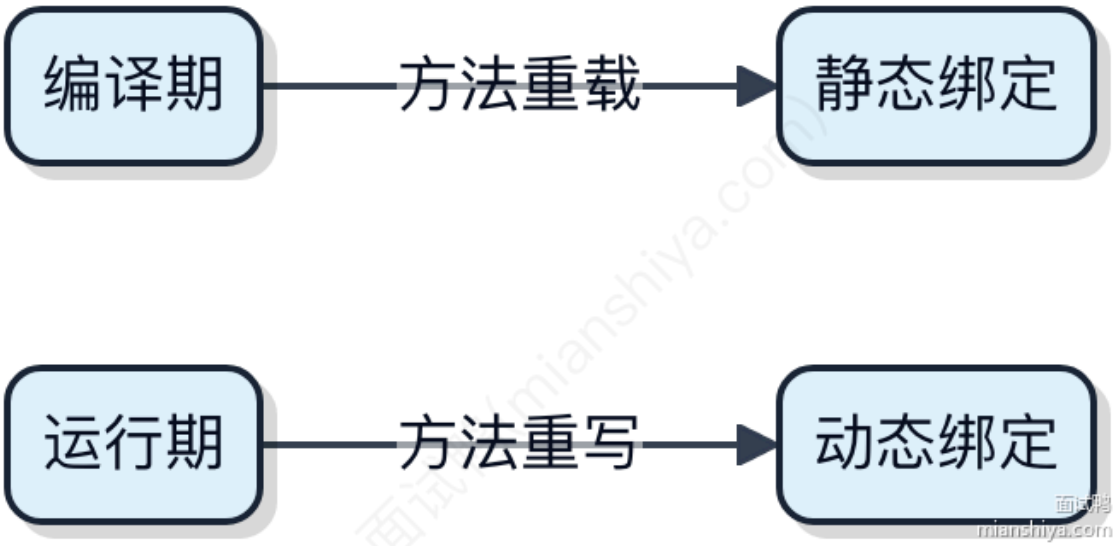
1）方法重载发生在同一个类里，靠的是参数列表的不同。比如 `Math.max(int, int)` 和 `Math.max(double, double)` ，返回类型可以不一样，但光靠返回类型不同是搞不定重载的。它在编译期就决定了调用哪个方法，属于**编译时多态**。

2）方法重写是子类对父类方法的覆盖，要求方法名、参数列表、返回类型都一致（子类返回类型可以是父类的子类型），而且访问权限不能更严格。它是运行时才确定调用哪个版本，比如 `ArrayList.toString()` 覆盖了 `Object.toString()` ，这就是**运行时多态**的体现。

代码上看：

```
class Parent {
    void show() { }
}
class Child extends Parent {
    @Override
    void show() { } // 重写
}
```

重载更像是一个类内部的多个同名工具方法，像 `Arrays.sort()` 就有好几种参数形式；而重写是为了实现多态，让 `List<String> list = new ArrayList<>()` 调用 `list.add()` 时，实际执行的是 `ArrayList` 的逻辑。



Java17 有哪些新特性？

Java 17 是一个长期支持（LTS）版本，直接从 Java 11 跳过来的，所以积累了不少实用更新。重点不是语法大改，而是让代码更安全、更简洁、运行更高效。

1）**密封类（Sealed Classes）**
想限制一个类只能被指定的几个子类继承，用 `sealed` 关键字就行。以前靠文档约定，现在编译器能强制校验。比如你定义 `Shape` 只能由 `Circle` 、 `Rect` 实现，别的类继承直接报错。

```
public sealed interface Shape permits Circle, Rect {}
public record Circle(double r) implements Shape {}
public final class Rect implements Shape {}
```

2）**switch 模式匹配（预览功能）**
虽然 Java 17 还是预览版，但已经能看出趋势。以前 switch 里要先判断类型再强转，现在一步到位。

```
switch (obj) {
    case String s -> System.out.println("字符串: " + s);
    case Integer i -> System.out.println("数字: " + i);
    default -> System.out.println("其他");
}
```

3）移除了 Applet API，彻底跟老旧浏览器插件说再见。同时 ZGC 和 Shenandoah 在这个版本已经可用，大堆内存（比如 1TB）下也能把 GC 停顿压到 10ms 以内，适合对延迟敏感的服务。

4）默认启用弹性元空间，减少 Full GC 中元空间回收的开销。底层实现上，把很多原来用 C++ 写的脏活累活交给了 Java，维护起来更方便。

总的来说，Java 17 更像是“成熟期”的一次加固，**密封类**和模式匹配这些特性，都在引导你写出更可读、更少出错的代码。升级后一般不需要改业务逻辑，但能明显提升系统稳定性和可维护性。

Float 经过一系列的操作后(加减乘除)，如何判断是否和另一个数相等呢？

浮点数相等判断是个经典坑，根本原因在于二进制无法精确表示所有十进制小数。比如 `0.1` 在二进制里是无限循环的，存的时候就存在舍入误差，经过几次运算后误差累积，直接用 `==` 判断等于基本会翻车。

解决办法是用“误差范围”来判断，也就是看两个数的差值是否足够小。这个足够小的值通常称为 **epsilon**，Java 里可以借助 `Math.ulp()` 或者直接定义一个极小阈值。

```
float a = 0.1f * 3;
float b = 0.3f;
// 错误做法
// if (a == b) // 可能为 false

// 正确做法
if (Math.abs(a - b) < 1e-6) {
    // 认为相等
}
```

实际业务中，金融计算压根不会用 `float` 或 `double` ，而是上 `BigDecimal` 。像支付宝、银行系统这些对精度要求高的场景，都是 `BigDecimal` 在扛住，因为它是基于十进制的精确计算。

有个细节是， `1e-6` 这种阈值不是万能的，对于很大或很小的数值可能不适用。更稳妥的做法是结合相对误差：

```
public static boolean floatEquals(float a, float b) {
    return Math.abs(a - b) <= Math.max(Math.ulp(a), Math.ulp(b));
}
```

总之，浮点数判等别用 `==` ，要么用误差容忍比较，要么直接上 `BigDecimal` 做精确计算。

Java25 有哪些新特性？

Java 25 是一个短期支持版本（仅维护半年），更多是为长期版本探路，真正值得关注意的是它背后的实验性功能和语言演进方向。

1）**虚拟线程（Virtual Threads）进入第二轮预览**
这是 Project Loom 的核心成果，目标是让高并发编程变得简单。传统线程成本高，每个线程占用 MB 级栈内存，最多开几万条就到头了。而虚拟线程由 JVM 调度，轻量到可以同时跑百万级，像 Tomcat 的 NIO 处理模型可以直接用同步代码写出来。

```
Thread.startVirtualThread(() -> System.out.println("Hello, Loom"));
```

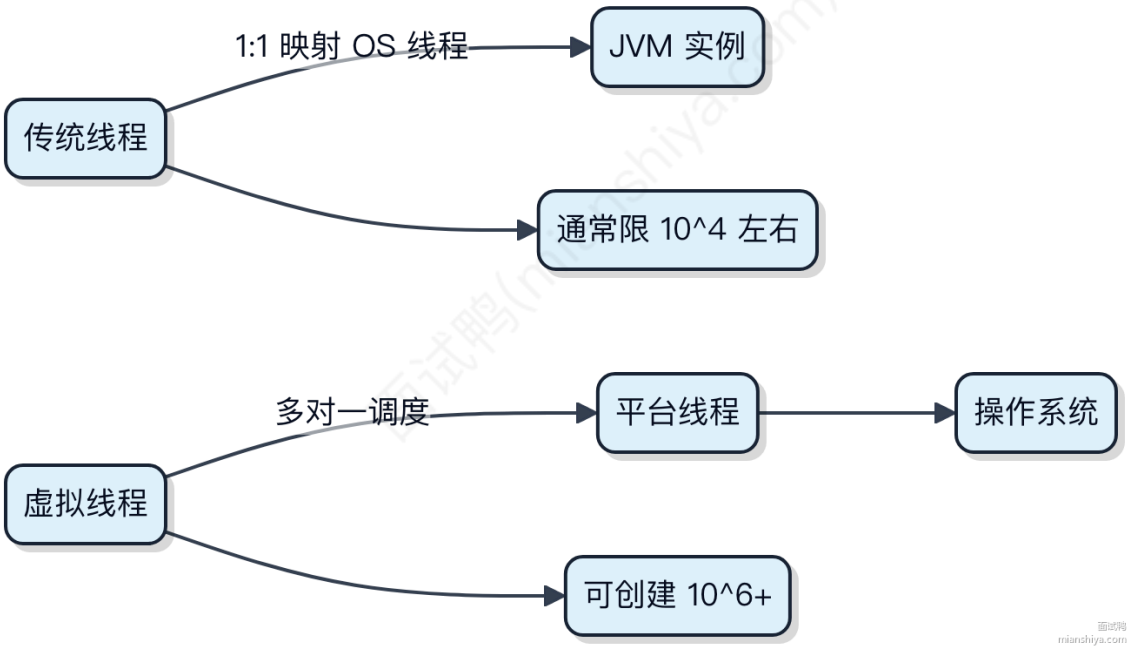
2）**未命名变量和模式（Unnamed Variables and Patterns）**
当你写模式匹配或 lambda 时，有些变量根本不用命名，比如只关心集合结构不关心内容，现在可以用下划线 `_` 占位，代码更干净。

```
if (obj instanceof String _) { /* 只判断类型 */ }
```

3）**外部函数与内存 API（Foreign Function & Memory API）再次孵化**
允许 Java 直接调用 native 库，替代老旧的 JNI。你可以像 Go 或 Rust 那样操作堆外内存、调用 C 函数，性能更好且更安全。比如用这个 API 调 SQLite 原生接口，延迟能压到微秒级。

4）其他小更新包括：ZGC 支持提前终止、权限默认禁用、record 类扩展等，但都不是生产级强需求。

真正影响深远的是虚拟线程，一旦稳定，整个中间件生态都会重构，像 Netty、Spring WebFlux 的异步模型可能会被同步 + 虚拟线程取代。目前建议在测试环境试水，别直接上生产。



Java11 有哪些新特性？

Java 11 是一个长期支持（LTS）版本，直接接替 Java 8，带来不少实用改进。它不光是语法升级，更多是精简和增强平台能力。

1）**HTTP Client 标准化**
原生 `HttpClient` 支持同步异步请求，WebSocket 也一并支持。以前得靠第三方库比如 OkHttp，现在 JDK 自带就能干。

```
HttpClient.newHttpClient()
    .sendAsync(request, BodyHandlers.ofString())
    .thenApply(Response::body)
    .thenAccept(System.out::println);
```

2）**字符串操作更顺手**
`String` 新增了 `isBlank()`、`lines()`、`strip()`（比 `trim` 更智能，处理 Unicode 空格）、`repeat(n)`。日常处理文本省了不少事。

3）**文件读写一行搞定**
`Files.readString()` 和 `Files.writeString()` 支持 UTF-8 编码的一次性读写，小文件场景不用再套 `BufferedReader` 套路了。

```
String content = Files.readString(Path.of("data.txt"));
```

4）**运行单文件源码**
可以直接 `java HelloWorld.java` 运行，不用先 `javac`。适合脚本类场景或教学演示，开发调试更轻量。

5）**ZGC 初登场**
虽然 ZGC 在 Java 11 还是实验性功能，但它主打“低延迟”，目标是停顿时间不超过 10ms，能扛住 TB 级堆内存。后来在 Java 15 转正。

6）**移除部分旧内容**
彻底移除了 Java EE 和 CORBA 模块（比如 `javax.xml.ws`），官方明确这些技术已经过时，推荐用 Spring Boot 或 Micronaut 替代。

总的来说，Java 11 把 **实用性** 和 **现代化** 推进了一大步，尤其是 HTTP 客户端和字符串 API，基本成了新项目的标配底座。

Java Object 类中有什么方法，有什么作用？

每个 Java 类都默认继承 Object，它提供的方法是 JVM 和语言层面协作的结果。这些方法构成了对象行为的基础，像 equals、hashCode 这种在集合类里天天用。

1） `equals(Object obj)` 判断两个对象是否逻辑相等。默认实现是 == 比较，但像 String、Integer 都重写了它，按值比较。注意要满足自反、对称、传递等特性。

2） `hashCode()` 返回对象的哈希码。**散列表**比如 HashMap、HashSet 就靠它定位桶位置。重写 equals 时必须重写 hashCode，不然 HashMap 里可能找不到你存的对象。

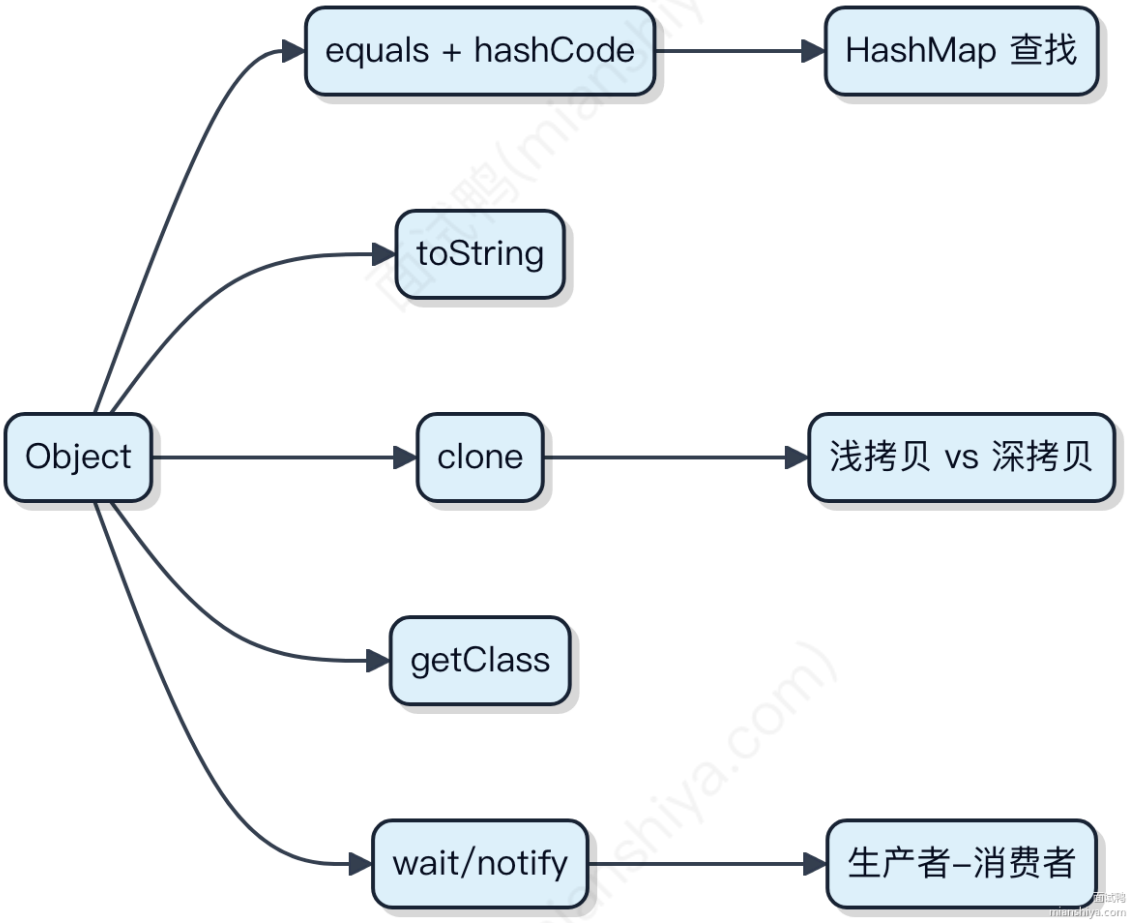
3） `toString()` 返回对象的字符串表示。打印对象或字符串拼接时自动调用。建议所有类都重写它，不然输出像 `java.lang.Object@6504e3ll`，看不出内容。

4） `clone()` 创建并返回对象的拷贝。要实现 Cloneable 接口，否则抛异常。浅拷贝只复制基本类型和引用地址，深拷贝得自己递归处理。

5） `getClass()` 返回运行时类对象，final 方法不能被重写。它是反射的入口，比如 Class.forName 就能拿到类信息。

6） `wait()` 、 `notify()` 、 `notifyAll()` 配合 synchronized 实现线程间通信。wait 会释放锁并挂起线程，直到被 notify 唤醒。用的时候必须在同步块里。

7） `finalize()` 对象被回收前可能调用的方法。但不保证执行，也不推荐用，资源释放应该用手动 close 或 try-with-resources。



接口和抽象类有什么区别？

接口和抽象类虽然都能定义方法契约，但它们的定位和能力完全不同。

抽象类本质还是个类，它允许你写具体的方法实现，也能定义成员变量。子类继承它的时候，可以复用里面已有的代码。比如你有个 `BaseService` 抽象类，里面写了通用的日志记录逻辑，所有业务服务继承它就能直接用。

接口就更纯粹了，从 Java 8 开始允许默认方法实现，但它主要还是用来声明“能做什么”。一个类可以实现多个接口，比如 `ArrayList` 既实现了 `List` 又实现了 `RandomAccess` 。这在设计扩展点时特别有用，像 Spring 的 `ApplicationContextAware` 、 `InitializingBean` 都是通过接口让 Bean 感知容器生命周期。

1）抽象类强调“是什么”，适合有共同属性和行为的场景
2）接口强调“有什么能力”，适合解耦和横向扩展
3）抽象类只能单继承，接口可以多实现

```java
public abstract class Animal {
    protected String name;
    public abstract void makeSound();
    public void sleep() {
        System.out.println("Sleeping...");
    }
}

public interface Flyable {
    default void fly() {
        System.out.println("Flying...");
    }
}
```

实际开发里，一般用抽象类做骨架实现，接口定义行为规范。像 JDK 里的 `InputStream` 是抽象类，因为有共用的读取逻辑；而 `Comparable` 就是典型的能力接口。

Java 中的序列化和反序列化是什么？

序列化就是把内存里的对象变成字节流，方便存储或传输。反序列化则是把这个字节流重新还原成对象。

1） 要让一个类支持序列化，必须实现 `Serializable` 接口，这个接口是个标记接口，不带任何方法。JVM 会通过反射机制自动处理字段的读写。

```java
class User implements Serializable {
    private String name;
    private int age;
}
```

2） 序列化时会生成一个 `serialVersionUID` ，用来校验版本一致性。如果反序列化时类结构变了，但 `serialVersionUID` 匹配，就能成功加载，否则抛 `InvalidClassException` 。建议显式定义它，避免因字段变动导致意外失败。

3） 静态变量和被 `transient` 修饰的字段不会被序列化。比如密码这类敏感信息，可以用 `transient` 标记，让它绕过持久化过程。

场景上，RMI、Dubbo 的网络调用底层就依赖序列化传对象。Redis 存 Java 对象时也常做序列化，比如用 JdkSerializationRedisSerializer。

不过默认的序列化性能差，产生的字节流大，跨语言也搞不定。实际项目里更多用 **Kryo**、**Protobuf** 或 **JSON**（如 Jackson）替代。

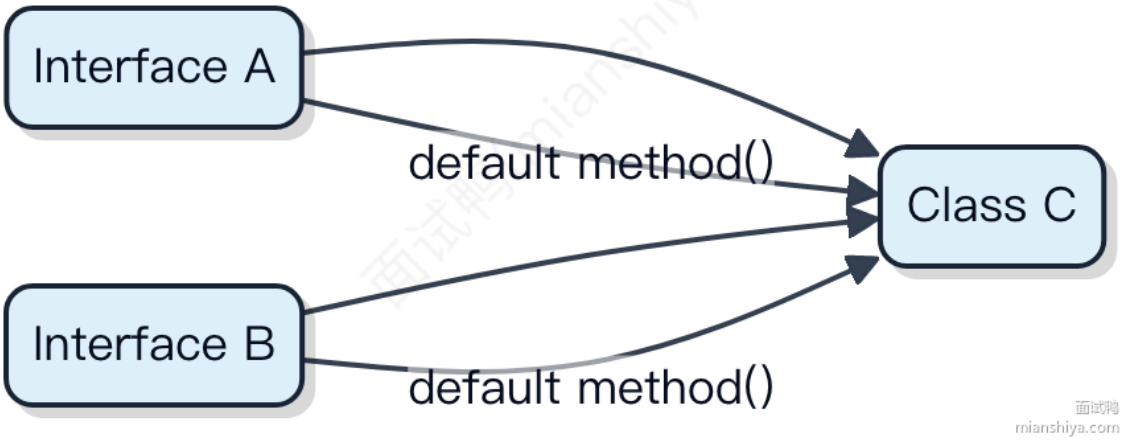
为什么 Java 不支持多重继承？

Java 不支持类的多重继承，主要是为了避免“菱形问题”带来的歧义和复杂性。想象两个父类有同名方法，子类继承时就不知道该调用哪个，编译器压根没法决定。

1） 类只能单继承，这是 Java 语言设计时的明确选择，保证继承链清晰。每个类有且只有一个直接父类，Object 是所有类的最终祖先。

2） 接口可以多继承，从 Java 8 开始，接口允许默认方法，解决了部分多重行为复用的需求。比如一个类实现多个接口，每个接口提供默认实现，只要方法签名不冲突就没问题。

3） 如果多个接口有同名默认方法，编译器会报错，必须由子类显式重写该方法，明确指出逻辑如何处理。这样就把决策权交给开发者，而不是让系统猜测。



代码上，你得手动解决冲突：

```
class C implements A, B {
    @Override
    public void method() {
        // 必须重写，可以选择调用 A 或 B 的默认实现，或者自己写
        A.super.method();
    }
}
```

说白了，Java 用“**单继承 + 多接口**”的组合，既规避了复杂性，又保留了灵活性。像 Spring 框架里大量依赖接口做解耦，就是这种设计的实际受益场景。

Java 运行时异常和编译时异常之间的区别是什么？

Java 的异常体系设计是为了让程序在出错时能有清晰的处理路径。我们平时说的运行时异常和编译时异常，其实是 **Exception** 体系下的两个分支，关键区别在于“编译器是否强制你处理”。

运行时异常，也就是 **RuntimeException** 及其子类，比如 **NullPointerException**、**ArrayIndexOutOfBoundsException**，这类异常编译器不强制你在代码里 try-catch 或 throws。它们通常是由程序逻辑错误导致的，比如空指针、数组越界。你可以处理，但不是必须的。

而编译时异常，像 **IOException**、**SQLException** 这些，只要方法里可能抛出，你就得显式处理，要么 catch，要么继续往上 throws。否则，代码根本过不了编译。这类异常往往是外部因素引起的，比如文件不存在、网络断开，属于“可预期但不可控”的问题。

简单说，编译时异常是编译器逼你面对的问题，你不写处理逻辑就编译不过；运行时异常是你自己代码的“bug”，编译器默认你已经校验好了，出了问题你自己负责。

```java
// 编译时异常：必须处理
FileInputStream fis = new FileInputStream("a.txt"); // 不处理会编译失败

// 运行时异常：可以不处理
int[] arr = new int[5];
arr[10] = 1; // 数组越界，程序直接崩，但编译没问题
```

所以设计上，如果你希望调用者必须考虑某种错误场景，就用编译时异常；如果是程序内部逻辑问题，用运行时异常更合适。现在很多框架比如 Spring，也倾向于用运行时异常来减少模板代码。

如何在 Java 中调用外部可执行程序或系统命令？

Java 里执行外部命令，主要靠 `Runtime.exec()` 或 `ProcessBuilder` 。虽然都能干这活，但后者更灵活，是现在推荐的方式。

1）用 `Runtime.exec()` 最简单，一行代码就行

```java
Process p = Runtime.getRuntime().exec("ls -l");
```

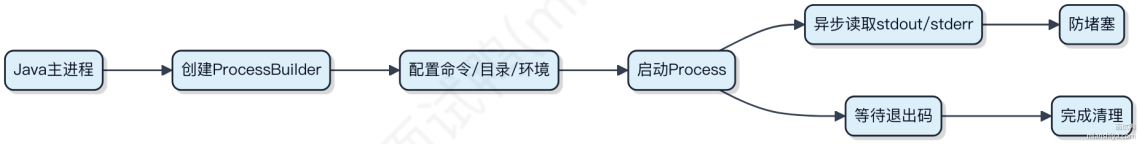
但它对环境变量、工作目录这些控制很弱，搞复杂场景容易翻车。

2） `ProcessBuilder` 就是为了弥补这个短板设计的，能精细控制命令的执行环境

```java
ProcessBuilder pb = new ProcessBuilder("ping", "baidu.com");
pb.directory(new File("/tmp")); // 指定工作目录
pb.inheritIO(); // 输出直接打到控制台，方便调试
Process p = pb.start();
```

真正关键的是**别忘了处理输入输出流**。子进程的 stdout 和 stderr 如果不读，缓冲区满了就会卡住，程序直接僵死。常见写法是用线程异步消费这两个流。

另外， `Process.waitFor()` 会阻塞等命令结束，返回退出码，0 一般是成功。超时控制用 `waitFor(long, TimeUnit)` ，避免无限等下去。



栈和队列在 Java 中的区别是什么？

栈和队列最根本的区别在于数据访问的顺序。

1）栈是后进先出（LIFO），就像一摞盘子，只能从顶部拿或放。Java 里可以用 `ArrayDeque` 模拟栈操作，比如调用 `push()` 入栈， `pop()` 出栈，取值永远在顶端。

2）队列是先进先出（FIFO），像排队打饭，新来的人排尾，前面的人先走。Java 中 `Queue` 接口定义了 `offer()` 入队， `poll()` 出队，处理顺序从头到尾。

实际开发中，**栈**常用于方法调用链、表达式求值，JVM 就靠虚拟机栈管理方法执行上下文。而**队列**多用于解耦生产消费，比如消息中间件 Kafka 的底层就是高性能队列在支撑，线程池的任务队列也是典型场景。

代码上对比也很直观：

```
ArrayDeque<Integer> stack = new ArrayDeque<>();
stack.push(1);
stack.push(2);
stack.pop(); // 返回 2

Queue<Integer> queue = new ArrayDeque<>();
queue.offer(1);
queue.offer(2);
queue.poll(); // 返回 1
```

注意别用过时的 `Stack` 类，它线程安全但性能差，直接用 `ArrayDeque` 做栈更高效。同样，普通场景用 `ArrayDeque` 实现队列，比 `LinkedList` 更快，内存更紧凑。

什么是 Java 的网络编程？

Java 的网络编程，说白了就是用 Java 代码实现不同机器上的程序能互相通信。最基础的就是基于 TCP 和 UDP 这两种协议来收发数据。

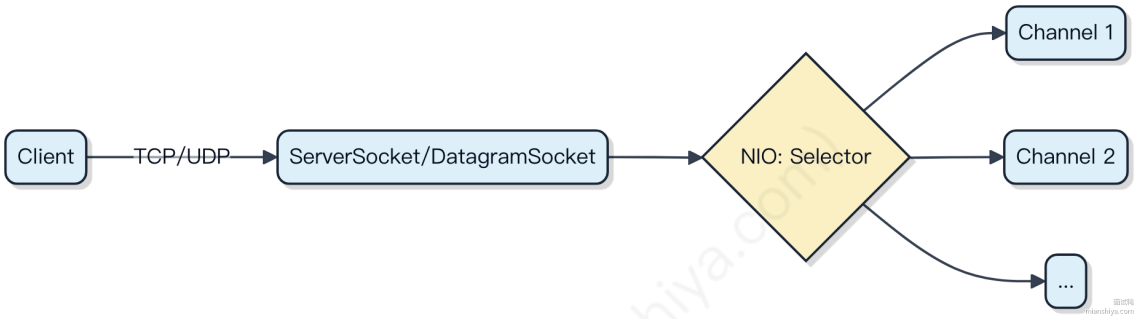
1）TCP 是面向连接的，比如你用 Socket 写个聊天程序，服务端得先启动 ServerSocket 监听端口，客户端再用 Socket 发起连接。一旦连上，双方就能通过输入输出流交换数据，保证可靠、有序。

```
// 服务端简单示例
ServerSocket server = new ServerSocket(8080);
Socket client = server.accept();
BufferedReader in = new BufferedReader(new
InputStreamReader(client.getInputStream()));
String msg = in.readLine(); // 读客户端消息
```

2）UDP 不建立连接，直接发数据包，适合对实时性要求高但能容忍丢包的场景，比如视频直播。Java 用 DatagramSocket 和 DatagramPacket 来处理。

3）实际开发中，没人会直接裸写这些底层 API。Netty 才是主流选择，它把 NIO 那套复杂的事件驱动模型封装得很干净，像 Dubbo、RocketMQ 都靠它撑起高性能通信。

NIO 和 **Netty** 是进阶关键，尤其是 Reactor 模式怎么通过一个线程管理成千上万个连接。传统 IO 每个连接一个线程，扛不住并发；NIO 用 Selector 监听多个通道，压根不经过阻塞等待。



什么是 Java 中的迭代器（Iterator）？

Java 里的迭代器（Iterator）就是专门用来遍历集合的一个对象，它让你能一个一个地访问集合里的元素，又不暴露底层结构。

1）核心方法就那几个：`hasNext()` 判断还有没有下一个，`next()` 拿出下一个元素，`remove()` 删除刚拿到的那个。比如遍历 ArrayList 或 HashSet 的时候，你用 foreach 其实背后就是 Iterator 在干活。

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String item = it.next();
    System.out.println(item);
}
```

2）它最大的好处是统一了遍历方式。不管你是 ArrayList 还是 LinkedList，TreeSet 还是 HashMap 的 key 集合，只要实现了 Iterable 接口，就能用同样的方式遍历。这其实就是**迭代器模式**的典型应用，解耦了集合和遍历逻辑。

3）fail-fast 机制也得提一嘴。像 ArrayList 的迭代器，如果遍历过程中别人修改了集合，它会直接抛 ConcurrentModificationException。这不是bug，是故意设计的，避免你在脏数据上操作。但如果你真要边遍历边删，记得用 `it.remove()`，别直接调集合的 remove。

4）有些场景不适合用普通 Iterator。比如你要并发遍历，就得考虑 CopyOnWriteArrayList 这种自带快照的结构，它的迭代器基于副本，改不影响读。

什么是 Java 的封装特性？

封装说白了就是把数据和操作数据的方法绑在一起，同时隐藏内部实现细节。你对外暴露的越少，别人就越难误用，后期修改也越安全。

比如一个银行账户类，余额这种关键数据不能让外部直接访问，得通过 `deposit` 或 `withdraw` 方法来操作。这些方法可以加校验逻辑，防止余额被随意篡改。

```
public class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
```

```
    public double getBalance() {
        return balance;
    }
}
```

你看，balance 是 **private** 的，外部没法绕过 deposit 直接改数值。这就是封装的价值——把“改余额”这个脏活累活关在类里，自己控制流程。

实际开发中，像 Spring 的 Bean、MyBatis 的 Mapper 接口，都靠封装把复杂逻辑收在背后。你调个 service.save() 就行，不用管它背后开了事务、刷了缓存、发了消息。

1）属性私有化，用 getter/setter 控制访问
2）内部实现细节不暴露，比如集合用 ArrayList 还是 LinkedList 外部不关心
3）增强安全性，避免对象状态被非法破坏

该藏的藏，该露的露，这才是合格的封装。

BigDecimal 为什么能保证精度不丢失?

浮点数精度问题，根源在二进制表示上。像 `0.1` 这种十进制小数，在二进制里是无限循环的，`float` 和 `double` 存的时候就只能近似，计算多了误差就越滚越大。

BigDecimal 不一样，它根本不玩二进制浮点那一套。它把一个数拆成两部分来看：一个是**无符号整数**（用 `BigInteger` 存），另一个是**缩放因子**（scale），也就是小数点要往左移多少位。

比如 `new BigDecimal("0.1")` ，内部存的是 `1` 这个整数，scale 是 `1` ，意思是 `1 / 10^1` 。所有运算都基于整数算，最后再按 scale 恢复小数位置，中间压根不经过浮点计算，自然不会丢精度。

关键得用字符串构造。写 `new BigDecimal(0.1)` 就糟了，因为 `0.1` 这个 `double` 值传进去之前就已经失真了。必须用字符串，让 BigDecimal 从字符逐位解析，才能保证原始值准确。

```
// 对
BigDecimal bd = new BigDecimal("0.1");
// 错，0.1 已经是 double 的近似值了
BigDecimal wrong = new BigDecimal(0.1);
```

它适合金融、交易这种一分钱都不能错的场景，像支付宝、银行系统里金额计算基本都靠它。但代价是比 `double` 慢得多，内存占用也大，一般业务没必要上。

1）内部用整数 + scale 模式避开了二进制浮点误差
2）必须用字符串构造，避免 double 先失真
3）性能差，只在需要精确计算时用

什么是 Java 的 BigDecimal?

Java 里的 BigDecimal 不是简单用来存小数的，它是为了解决浮点数计算精度丢失问题而存在的。像 float 和 double 在做加减乘除时，经常会出现 0.1 + 0.2 不等于 0.3 这种情况，这在金融、交易、计费系统里是完全不能接受的。

BigDecimal 能精确表示小数，它的底层用 int 类型的 `scale` 表示小数位数，用 BigInteger 存数值，所以能保证任意精度的运算准确。比如余额计算、汇率转换、订单金额拆分这些场景，必须用 **BigDecimal**。

但注意，它不是万能的。构造时别用 `new BigDecimal(double)` ，因为 double 本身就不准，得用字符串构造：

```
BigDecimal amount = new BigDecimal("0.1");
```

做除法更要小心，比如 1 除以 3 是无限循环小数，不指定精度会抛异常：

```
BigDecimal result = a.divide(b, 4, RoundingMode.HALF_UP); // 保留4位，四舍五入
```

性能上，BigDecimal 比基本类型慢得多，毕竟是对象操作，还有十进制的对齐和舍入处理。高频计算场景得权衡精度和性能。

另外，比较两个 BigDecimal 别用 equals，因为它会比 scale。0.1 和 0.10 用 equals 判断是 false，应该用 compareTo：

```
if (a.compareTo(b) == 0) { // 正确的值比较
    // 相等
}
```

Java 泛型擦除是什么？

Java 的泛型擦除指的是编译器在编译期把泛型信息拿掉，生成的字节码里压根不带类型参数。也就是说，`List<String>` 和 `List<Integer>` 到运行时都变成了 `List`，这个过程就叫**类型擦除**。

1） 编译器会在编译阶段检查泛型类型是否合法，比如你往 `List<String>` 里加整数，会直接报错。但一旦通过检查，就会把泛型信息擦掉，替换成对应的原始类型（raw type），比如 `List<T>` 变成 `List`，`T` 是引用类型的话默认用 `Object` 替代。

2） 如果泛型有上界，比如 `T extends Number`，那擦除后 `T` 就会被替换成 `Number`，而不是 `Object`。这样能保证方法调用的安全性。

3） 为了保证类型安全，编译器还会自动插入强制转换代码。比如你从 `List<String>` 取元素，虽然字节码是 `Object`，但编译器会自动加一句 `(String)` 转换。

```
List<String> list = new ArrayList<>();
list.add("hello");
String s = list.get(0); // 编译后实际是 (String) list.get(0)
```

4） 因为擦除发生在编译期，所以无法在运行时获取泛型的实际类型。这也是为什么不能 `new T()` 或 `if (obj instanceof List<String>)` —— 运行时根本没这信息。

有个例外是**反射**，如果泛型信息被保留在方法签名或字段上（比如作为成员变量或方法返回值），可以通过 `getGenericTypes()` 拿到，但这也只是“残留”的签名信息，不是运行时动态的。

Java 中的字节码是什么？

Java 源代码编译后生成的中间指令集，就是字节码。它不依赖具体硬件，运行在 JVM 上，实现“一次编写，到处运行”。

字节码文件以 `.class` 为后缀，内部是二进制格式，可以用 `javap -c` 反编译查看助记符形式的指令。比如一个简单的加法操作，会变成 `iconst_1`、`iconst_2`、`iadd` 这样的栈指令。

JVM 执行时，解释器逐条读取字节码并执行，热点代码会被 **即时编译器**（JIT）编译成机器码，提升性能。这个过程对开发者透明。

```
public class Add {
    public static int add() {
        return 1 + 2;
    }
}
```

反编译后你会看到：

```
0: iconst_1
1: iconst_2
2: iadd
3:ireturn
```

这说明 Java 的运算基于操作数栈，而不是寄存器。

字节码的好处在于可移植性，但这也带来了一层抽象开销。不过现代 JVM 通过 JIT 优化，大部分场景下性能接近原生代码。

像 Spring AOP 的动态代理、Lombok 编译期自动插入 getter/setter，都是在编译后修改字节码实现的。ASM、Javassist 这类库就是用来操作字节码的。

Java 和 Go 的区别

Java 和 Go 的设计哲学完全不同，一个追求企业级的稳重，一个追求现代并发的简洁。

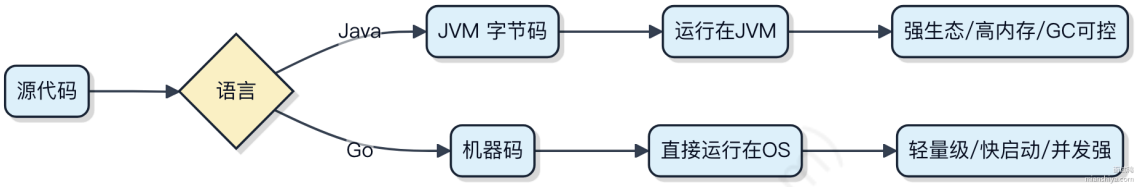
Java 走的是虚拟机路线，靠 JVM 实现跨平台，一套字节码跑在各种操作系统上。你写个 Spring Boot 服务，其实底层是 JVM 在做内存管理、GC、JIT 编译这些脏活累活。它适合大型系统，像阿里巴巴的电商架构、金融系统的交易核心，依赖丰富，生态庞大。

Go 直接编译成机器码，启动快，部署就是一个二进制文件，没有 JVM 依赖。它的 **goroutine** 是轻量级线程，用 channel 做通信，写并发服务特别爽。比如 etcd、Docker、Kubernetes 都是用 Go 写的，微服务网关、API 中台这类场景它能轻松扛住高并发。

语法上 Java 更啰嗦，接口、抽象类、泛型层层套娃，Go 就简单多了，没有继承，用组合和接口隐式实现，函数返回多值，错误处理靠显式判断。

性能方面，Java 有 JIT 优化，长期运行的服务性能很稳，但 GC 可能带来停顿。Go 的 GC 虽然也在进步，但目前还是更偏向低延迟，适合短平快的请求处理。

1）Java 适合复杂业务逻辑、已有生态深厚的大中型系统
2）Go 适合云原生、高并发、需要快速启停的微服务和基础设施



选哪个？看团队技术栈和场景。搞不定高并发微服务拆分，Java + Spring Cloud 也能扛，但要写个高性能代理网关，Go 几百行代码就能搞定。

[什么是 Java 中的动态代理？](#)

Java 的动态代理其实是在运行时，给一个对象生成代理类的技术。这个代理类会帮你把方法调用转发出去，典型的就是在不改原代码的情况下加日志、事务、权限控制这些逻辑。

JDK 自带的动态代理靠的是 `java.lang.reflect.Proxy` ，它要求被代理的对象必须实现至少一个接口。代理类在运行时创建，和目标对象实现同一个接口，然后把调用分发给 `InvocationHandler` 。

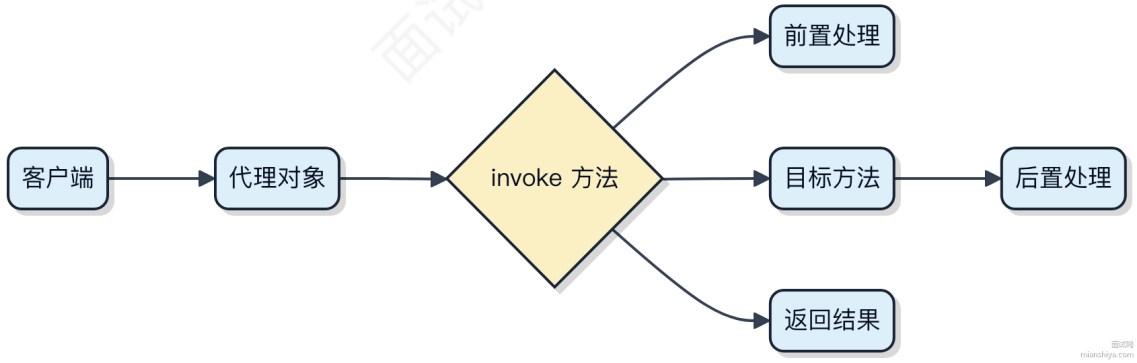
1）你写个类实现 `InvocationHandler` ，重写 `invoke` 方法，在里面控制实际的方法调用 2）用 `Proxy.newProxyInstance()` 生成代理实例，传入类加载器、接口数组和 handler 3）调用代理对象的方法时，全部会走到 `invoke` 里

比如 Spring AOP 在接口场景下就用 JDK 动态代理，像加个事务注解，方法执行前后自动开启提交事务。

代码长这样：

```
Object proxy = Proxy.newProxyInstance(
    target.getClass().getClassLoader(),
    target.getClass().getInterfaces(),
    (proxy, method, args) -> {
        System.out.println("前置逻辑");
        return method.invoke(target, args);
    }
);
```

还有一种是 CGLIB 动态代理，基于字节码生成子类，能代理普通类，Spring 内部混合使用这两种。**动态代理**的关键就是运行时生成，不用提前写死代理类。



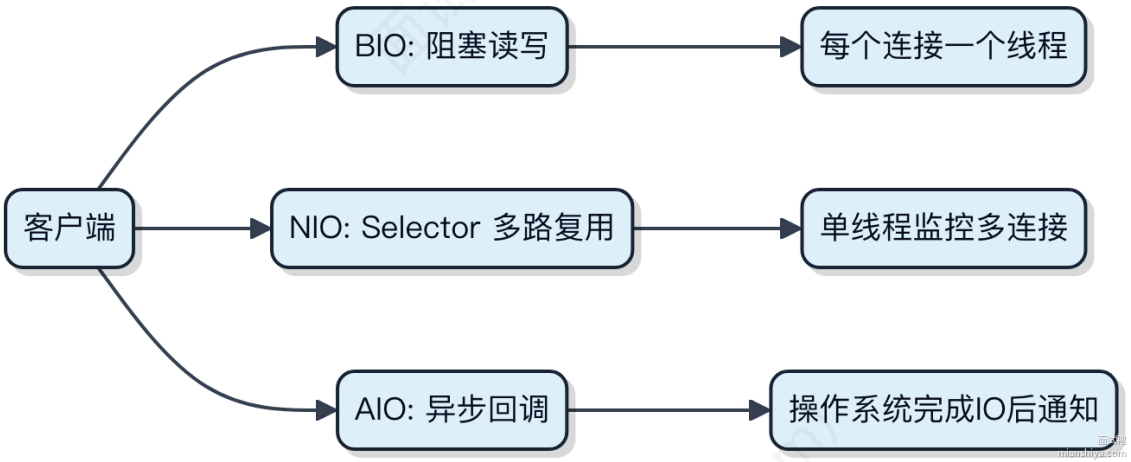
什么是 BIO、NIO、AIO?

BIO 就是传统阻塞 IO，每个连接配一个线程处理。像早期的 Tomcat 用 BIO 模式，1000 个客户端连上来就得开 1000 个线程，系统压根扛不住。线程一多上下文切换开销爆炸，资源消耗大。

NIO 的核心是**多路复用**，通过 Selector 统一管理多个连接。一个线程就能轮询上千个连接的状态，有数据才去读写。Java 里的 NIO 基于 epoll（Linux）或 kqueue（macOS），Netty 就是典型的 NIO 框架，能轻松支撑几十万并发。

AIO 更进一步，是真正的异步非阻塞。读写操作由系统内核完成，完成后通知程序回调。比如读文件时发起请求就直接返回，等数据准备好了操作系统主动告诉你。Windows 的 IOCP 是典型实现，但 Java 中 AIO 使用较少，主要因为编程模型复杂，且 Linux 对 AIO 支持不如 epoll 成熟。

1）BIO 适合连接数少、业务耗时长的场景，比如内部工具服务
2）NIO 适合高并发、短消息的场景，主流都是它，像 Dubbo、RocketMQ 内部通信都基于 NIO
3）AIO 理论性能最好，但实际落地难，目前更多用 NIO + 多线程模拟异步



什么是 Java 的多态特性?

多态是 Java 面向对象的三大特性之一，它让同一个行为在不同对象上有不同的实现方式。说白了，就是“一个接口，多种实现”。

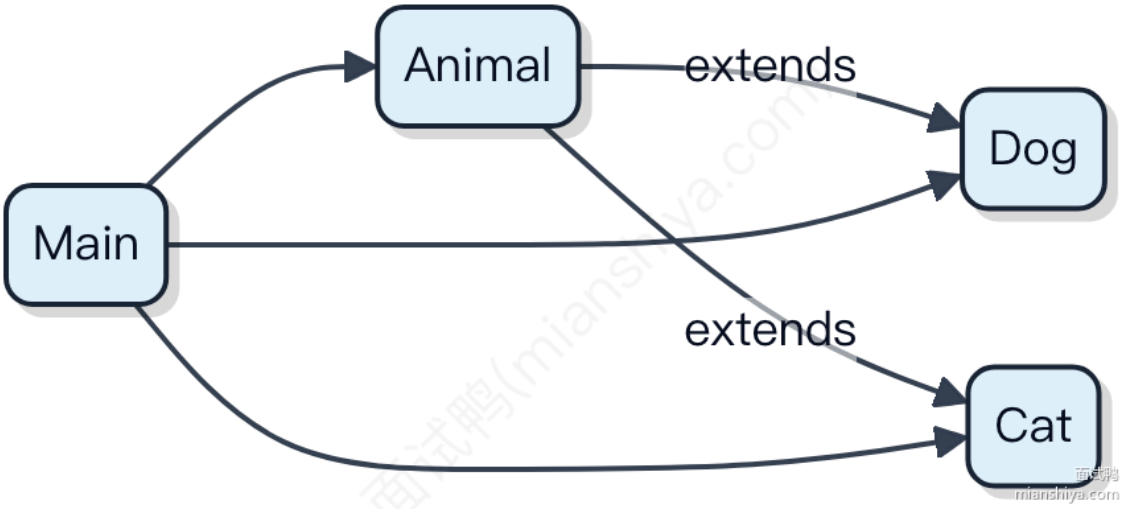
1）编译时多态
主要靠方法重载（overload）实现。同一个类里，方法名一样但参数列表不同，编译器在编译阶段就能确定调用哪个方法。

2）运行时多态
靠方法重写（override）和继承来实现。父类引用指向子类对象，调用被重写的方法时，实际执行的是子类的版本。这个绑定过程发生在运行时，由 JVM 动态决定。

```
Animal a = new Dog();
a.makeSound(); // 调用 Dog 的 makeSound
```

这种机制的核心在于**动态分派**，JVM 会根据对象的实际类型去方法区找对应的实现，而不是看引用的声明类型。这也是为什么多态能支持灵活的扩展性——上层代码只需要依赖抽象，比如 Spring 中的 Bean 处理、MyBatis 的插件链，都大量利用了这一特性。

不过要注意，静态方法、private 方法、构造方法不参与多态，它们的调用在编译期就定死了。



这样设计的好处是解耦。新增动物类型不用改原有逻辑，只要继承 Animal 实现 makeSound 就行，系统更容易维护和扩展。

Java 中的参数传递是按值还是按引用？

Java 里所有参数传递都是按值传递，不存在按引用传递。这个“值”指的是变量副本，具体分两种情况。

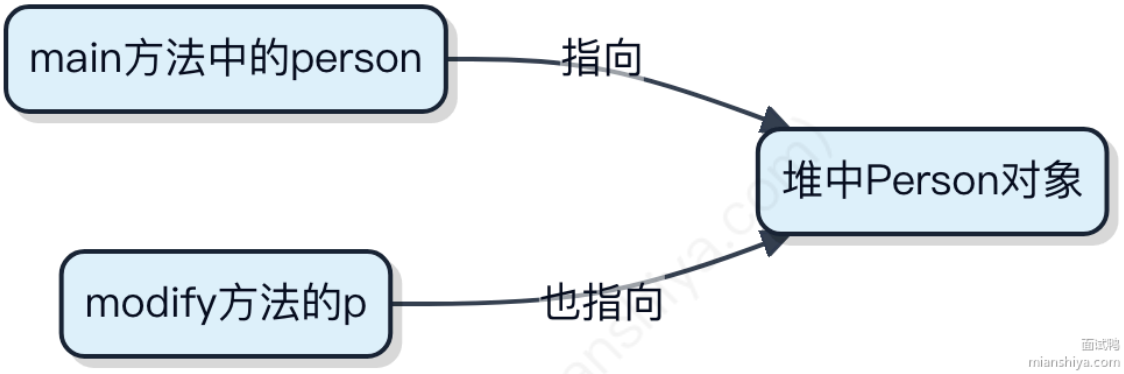
基本类型传的是数据副本，比如 int、boolean，方法内部改了参数，外面的变量完全不受影响。你改你的，我这儿还是原来的值。

对象类型传的是**引用的副本**，也就是说，两个变量指向同一个堆内存对象。方法内部通过这个参数修改对象的字段，外部能看得到，因为操作的是同一个实例。但如果你在方法里给参数重新赋值，比如 new 一个新对象，那只是改变了参数副本的指向，原始变量依然指向旧对象。

```
void modify(Person p) {
    p.name = "new";       // 外部可见，改的是共享对象
    p = new Person();     // 外部不可见，只是参数副本换了指向
}
```

很多人混淆是因为看到对象内容能被改，就以为是引用传递。其实关键在于：参数本身是传了个拷贝，方法没法改变原变量的指向。String 这种不可变类更明显，任何“修改”都会生成新对象，原字符串压根不变。

1）基本类型：传值，互不影响
2）对象类型：传引用的副本，能改对象内容，不能改原引用指向



什么是 Java 中的不可变类？

不可变类指的是实例一旦创建，其内部状态就无法被修改的类。Java 里的 String 就是最典型的例子，你对字符串做拼接，其实是生成了新对象，原字符串并没变。

要写一个真正的不可变类，得把几个关键点都守住。首先是类本身要用 final 修饰，防止被继承破坏规则。然后所有字段必须是 private 且用 final 修饰，确保外部不能直接改，初始化后也不能再赋值。

1） 所有字段在构造函数里完成初始化，而且必须深拷贝，特别是集合或数组这种引用类型，否则外部拿到引用还是能改里面的内容。

2） 不提供任何 set 方法或能修改内部状态的 public 方法。

3） 如果有返回可变字段的方法，比如返回一个 List，那得用 Collections.unmodifiableList 包一层，防止外部通过返回值去修改内部数据。

```
public final class ImmutableUser {
    private final String name;
    private final List<String> tags;

    public ImmutableUser(String name, List<String> tags) {
        this.name = name;
        this.tags = new ArrayList<>(tags); // 深拷贝
    }

    public String getName() {
        return name;
    }

    public List<String> getTags() {
        return Collections.unmodifiableList(tags); // 只读视图
    }
}
```

这种设计在多线程环境下特别省心，因为状态不会变，天然线程安全，像 ConcurrentHashMap 的 key 就推荐用不可变对象。但代价是每次“修改”都要新建对象，频繁变更的场景可能产生大量临时对象，GC 压力会大。

Java 中 Exception 和 Error 有什么区别？

Java 里 Exception 和 Error 都继承自 Throwable，但代表的完全是两类问题。

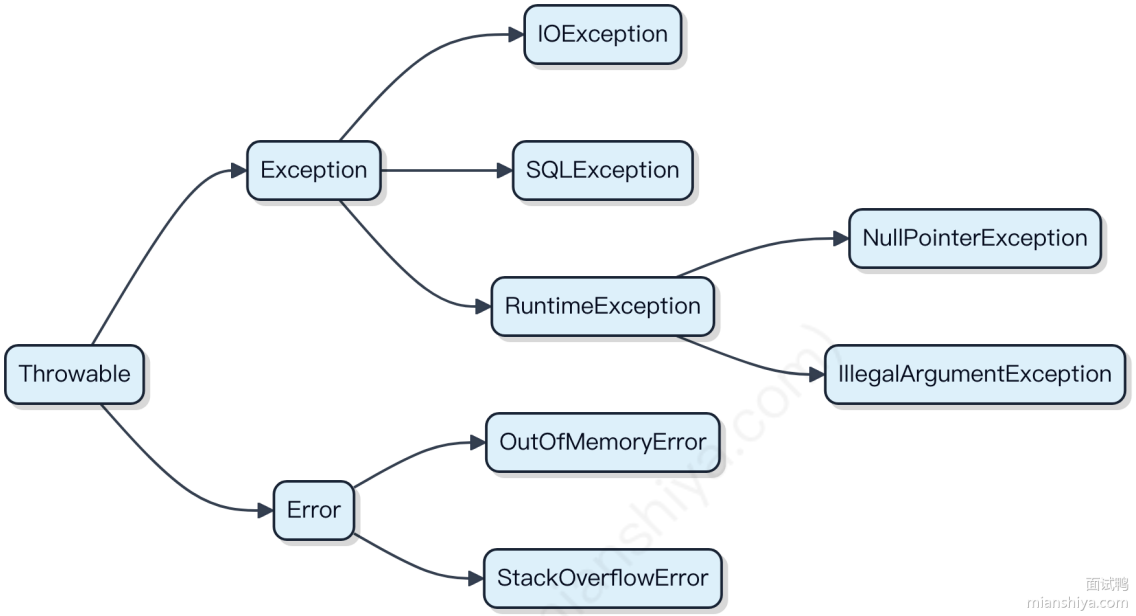
Exception 指程序能预见并处理的异常情况，比如文件没找到、网络超时、数组越界。这类问题通常可以通过代码逻辑恢复，像 try-catch 捕获后重试或降级。常见的 IOException、SQLException 都属于这一类。

Error 则代表 JVM 自身出了严重问题，比如内存溢出（OutOfMemoryError）、栈溢出（StackOverflowError）、类加载失败（NoClassDefFoundError）。这些问题程序本身搞不定，即使捕获也很难恢复，一般会导致应用崩溃。你写业务代码几乎不需要去 catch Error。

1）Exception 分为 checked 和 unchecked。checked 异常必须显式处理，比如在方法签名上 throws，或者用 try-catch 包住。unchecked 异常（即 RuntimeException 及其子类）则不用强制处理。

2）Error 和 RuntimeException 都属于 unchecked，编译器不强制你处理。

3）实际开发中，服务间调用超时、数据库连接失败这些该做重试或熔断，属于 Exception 的处理范畴；而一旦出现 OOM，整个进程可能已经不稳定，这时候最好的做法反而是快速失败，让监控告警，而不是试图“修复”。



Java 面向对象编程与面向过程编程的区别是什么？

面向过程是写一堆函数，数据散在外面，谁要用谁就去调。

Java 的面向对象不是简单把函数打包，而是把**数据和操作数据的方法绑在一起**，形成一个独立的“对象”。比如用户信息和它的验证逻辑、更新逻辑都封装在 User 类里，外面只能通过暴露的方法来交互。

1）你改内部实现，只要接口不变，外部压根不感知。
2）不同对象之间通过方法调用协作，像拼积木一样搭系统。
3）继承和多态让你能复用代码，也能做到运行时动态替换行为。

举个例子，处理订单流程，面向过程可能是一连串函数：checkStock → calcPrice → createOrder → sendSms。数据在各个函数间传来传去，一改结构就得动全部。

而面向对象会有一个 OrderService，它依赖 Inventory、Pricing、Notification 这些对象，每个对象自己管自己的状态和逻辑。调用方只关心“下单”这个动作，背后怎么协调是它自己的事。

代码上差别的关键是抽象层级：

```
// 面向过程风格
void processOrder(long userId, List<Item> items) {
    if (!Inventory.check(items)) throw new IllegalStateException();
    double price = Pricing.calc(items);
    long orderId = DB.saveOrder(userId, items, price);
    Notification.send(userId, "Order " + orderId + " created");
}

// 面向对象风格
orderService.placeOrder(user, items); // 内部流转，对外透明
```

说白了，面向对象是靠**封装、职责划分和消息传递**来管理复杂度，适合 10w+ 行以上的系统。小脚本搞个 main 函数一路写到底，其实也没问题。

什么是 Java 内部类？它有什么作用？

内部类就是定义在另一个类里面的类。它能直接访问外部类的所有成员，包括私有的，这种紧密的耦合关系在某些场景下特别有用。

1）解决逻辑相关的类组织问题。比如 `LinkedList` 里的 `Node`，作为链表的节点结构，天生属于 `LinkedList` 的内部实现细节，用内部类封装就很自然。
2）实现事件监听、回调等机制时更方便。像 AWT/Swing 的事件处理，监听器作为内部类可以直接拿到外部界面组件的状态，不用到处传引用。
3）替代函数式接口或简化匿名类写法。虽然现在有 Lambda，但在需要维护状态的回调里，局部内部类还是更灵活。

匿名内部类在创建线程或注册监听时很常见：

```
new Thread(new Runnable() {
    public void run() {
        System.out.println("来自内部类的线程");
    }
}).start();
```

注意内存泄漏风险。非静态内部类会隐式持有外部类引用，如果它的实例生命周期比外部类长（比如被缓存了），就会导致外部类无法回收。Android 开发里经常踩这个坑。

静态嵌套类没有这个问题，它和普通类差不多，只是命名空间被包装了一下：

```
public class Outer {
    static class Nested { } // 不依赖外部类实例
}
```

内部类最终会被编译成独立的 `.class` 文件，比如 `Outer$Inner.class`，JVM 其实并不认识“内部类”这个概念，全是靠编译器生成代码和桥接方法实现的。

Java8 有哪些新特性？

Lambda 表达式不是语法糖，它在字节码层面通过 invokedynamic 指令实现，运行时动态绑定调用点。写匿名内部类的代码量直接砍掉一大半。

方法引用让你用 `System::out::println` 这种写法替代 lambda，可读性提升明显，尤其在流操作里连贯性很强。

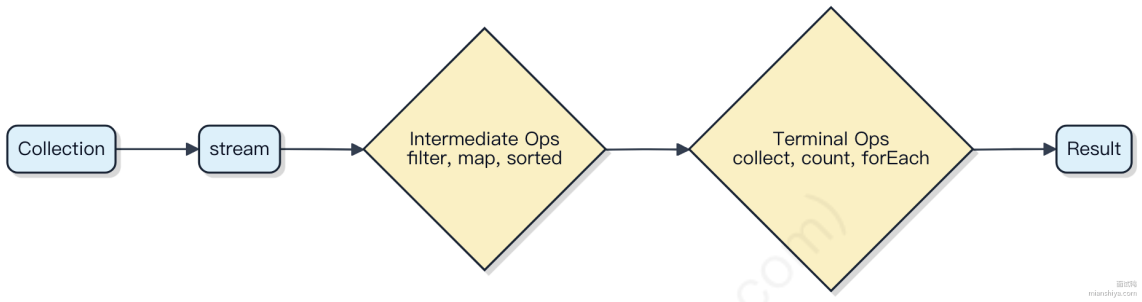
接口可以定义 default 方法，JDK8 里的 `Collection` 接口新增的 `stream()` 就是典型例子。接口演化不再破坏实现类，这点对库开发者太重要了。

Stream API 是集合处理的一次升级，filter、map、reduce 链式调用，逻辑表达更接近业务语言。比如 `users.stream().filter(u -> u.getAge() > 18).count()` 统计成年人，代码意图一眼就懂。

Optional 不是解决空指针的银弹，但它强制你显式处理 null 情况。 `isPresent()` 和 `ifPresent()` 配合使用，能避免随意调用可能为空的对象方法。

时间 API 全面重构， `LocalDateTime` 、 `ZonedDateTime` 、 `Duration` 这一套比原来 `Date` 和 `Calendar` 好用太多。线程安全，语义清晰，解析格式也统一交给 `DateTimeFormatter` 。

ConcurrentHashMap 在 JDK8 里数据结构变了，底层从分段锁改成 Node 数组 + 链表/红黑树，CAS + synchronized 控制并发，吞吐量明显提升。



这些特性里，Stream 和 Lambda 彻底改变了 Java 的编程风格，后续版本的函数式支持都在这基础上演进。

为什么 JDK 9 中将 String 的 char 数组改为 byte 数组？

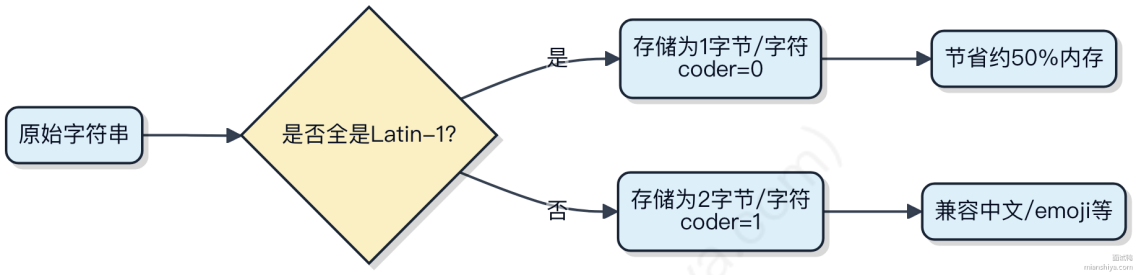
JDK 9 这个改动其实是为了解决字符串内存占用过高的问题。以前 `String` 用 `char[]` 存储，每个字符固定占 2 字节，但现实中大部分字符串比如英文、数字、符号，其实用 1 字节的 ISO-8859-1 或 UTF-8 就够了。

于是 JDK 9 引入了 **紧凑字符串**（Compact Strings）设计，底层改用 `byte[]` 加一个编码标识 `coder` 。如果是纯 Latin-1 字符，就用 1 字节存储；如果有中文、特殊字符等需要 UTF-16，才切到 2 字节模式。这样在普通业务场景下，字符串内存直接省了接近一半。

这个 `coder` 标志位决定了编码方式，取值通常是 0 表示 LATIN1，1 表示 UTF16。所有 String 操作都会先判断 coder 再处理数据，对开发者完全透明。

举个例子，像日志系统里大量路径、参数名、状态码这类文本，基本都是 ASCII，原来白白浪费一倍空间。现在 Logback、Spring Boot 这些框架跑在 JDK 9+ 上，堆内存压力明显降低。

当然也有代价。混合编码意味着每次访问都要判断 coder，会有少量性能损耗。但在绝大多数以读为主的场景下，空间换时间是划算的。



所以这波操作本质是 JVM 层面的空间优化，针对真实业务负载做的妥协和平衡。对于搞中间件或者调优的人来说，理解这点有助于分析堆 dump 时更清楚字符串的真实开销。

Java 中 String、StringBuffer 和 StringBuilder 的区别是什么？

字符串操作在 Java 里太常见了，这三个类总被拿来问。关键得说清楚它们的线程安全和性能差异。

String 是不可变的，每次拼接都会生成新对象，频繁操作就搞不定，比如用 + 循环拼字符串，底层其实是不断 new StringBuilder 再转回 String，效率低。

StringBuilder 是可变的，非线程安全，单线程下拼接字符串首选它，性能最好，append 操作就是直接往 char 数组后面加，扩容时一般是当前容量的 1.5 倍再加 1。

StringBuffer 和 StringBuilder 接口几乎一样，但它的方法都加了 synchronized，线程安全，适合多线程环境，但锁带来开销，性能比 StringBuilder 低大概 10%-20%。

1）如果字符串内容不变，用 String
2）单线程大量拼接，比如日志组装、JSON 构建，用 StringBuilder
3）多线程共享拼接场景，比如 Web 应用中多个请求共用一个缓冲区，才考虑 StringBuffer

代码上看区别：

```
// String 拼接，不推荐循环内使用
String s = "";
for (int i = 0; i < 1000; i++) {
    s += "a"; // 每次都 new 对象
}

// 正确做法
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append("a");
}
String result = sb.toString();
```

其实日常开发里，StringBuffer 已经很少见了，大多被 StringBuilder + 手动同步或 ConcurrentHashMap 这类结构替代。

Java 的 StringBuilder 是怎么实现的？

StringBuilder 的本质是个可变的字符数组，解决字符串拼接时频繁创建对象的问题。每次用 `+` 拼字符串，底层其实会 new 多个 String 和 StringBuilder，循环里搞这个，分分钟 OOM。

它内部维护一个 char[]，叫 **value**，初始容量是 16。你 append 字符，就是往这个数组里塞数据，有个 count 记当前长度。数组不够了，就扩容，新大小是原来 2 倍再加 2，然后用 System.arraycopy 搬数据。

```
StringBuilder sb = new StringBuilder();
sb.append("hello");
sb.append("world");
```

上面这段，全程只操作一个 char[]，压根不经过 GC，性能比 String 拼接高好几个量级。单线程下，拼接超过 3 次的字符串，基本都该用 StringBuilder。

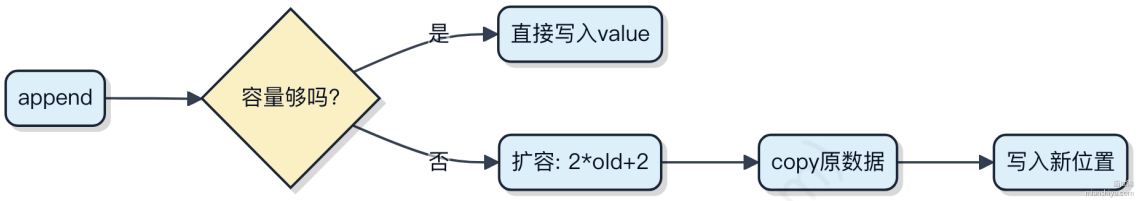
和 StringBuffer 的区别？后者所有方法都加了 synchronized，线程安全但慢。StringBuilder 就是它的非同步版本，99% 的场景都应该用它。

扩容机制要注意，如果预估不准容量，反复扩容拷贝数组，也会影响性能。比如一开始就 append 一个 1000 长度的字符串，最好指定初始大小：

```
new StringBuilder(1024)
```

避免后续多次扩容。

1）内部是可变 char[] 2）默认容量 16，扩容策略为 2 * old + 2 3）非线程安全，性能高，适合单线程拼接 4）大字符串拼接建议预设容量



Java 中包装类型和基本类型的区别是什么？

Java 里基本类型和包装类型最根本的区别在于，一个是栈上存的原始值，一个是堆上的对象实例。

1）基本类型像 int、boolean 这些直接在栈上分配，访问快，不涉及对象开销。比如 `int a = 1;` 就是个纯粹的 32 位整数。

2）包装类型是类，比如 Integer、Boolean，它们封装了基本类型，提供了工具方法，还能表示 null。但每次创建可能产生对象，有 GC 压力。比如 `Integer b = 1000;` 实际是 new Integer(1000) 的自动装箱。

自动装箱和拆箱在集合操作时特别常见。List 存的是对象，所以 int 会自动转成 Integer。但这里有个坑：-128 到 127 的 Integer 会被缓存，超出范围用 == 比较会出错。

```
Integer a = 127;
Integer b = 127;
a == b; // true，缓存命中

Integer c = 128;
Integer d = 128;
c == d; // false，两个不同对象
```

性能敏感场景优先用基本类型，避免频繁装箱拆箱带来的开销。而需要泛型、反射或允许 null 值时，就得用包装类。像 JSON 反序列化到字段，字段定义成 Integer 能区分“没传”和“传了0”。

装箱过程其实调的是 Integer.valueOf()，不是 new，所以能复用缓存对象。这个细节很多人忽略，但面试一问就露馅。

Java 中的 hashCode 和 equals 方法之间有什么关系？

重写 equals 方法时，必须同时重写 hashCode 方法，这是为了保证对象在哈希集合中的行为一致性。

1）如果两个对象通过 equals 比较返回 true，它们的 hashCode 必须相等。反过来，hashCode 相等，equals 不一定为 true，因为可能存在哈希碰撞。

2）比如你在 HashMap 里存一个自定义对象作为 key，map 会先通过 hashCode 找到桶位置，再用 equals 判断是否是同一个 key。如果你只重写了 equals 而没重写 hashCode，两个逻辑上相等的对象可能算出不同的 hash 值，导致 get 的时候压根不经过你 put 进去的那个桶，直接找不到。

3）反过来说，不重写 equals 只重写 hashCode 一般问题不大，但失去了自定义相等逻辑的意义。常见错误是只改了 equals 判断字段，比如 User 按 name 和 age 判等，但忘了同步更新 hashCode 计算逻辑。

代码示例：

```java
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof User)) return false;
    User user = (User) o;
    return age == user.age && Objects.equals(name, user.name);
}

@Override
public int hashCode() {
    return Objects.hash(name, age); // 字段要和 equals 保持一致
}
```

简单说，这两个方法得“绑定”着重写，不然像 HashSet、HashMap 这些依赖哈希行为的集合就会出错。

为什么在 Java 中编写代码时会遇到乱码问题？

Java 中的乱码问题，本质是**字符集不一致**导致的。你从哪读数据、往哪写数据，每个环节用的编码规则必须对得上，不然就是“鸡同鸭讲”。

比如你用 `UTF-8` 写了个中文文件，结果别人用 `GBK` 去读，那“你好”可能就变成“浣犬ソ”。反过来也一样。这种问题在 IO 操作、网络传输、数据库交互时特别常见。

1） 文件读写时，没指定编码，默认用了系统编码。Windows 一般是 `GBK` ，Linux/Max 是 `UTF-8` ，跨平台一跑，直接出事。

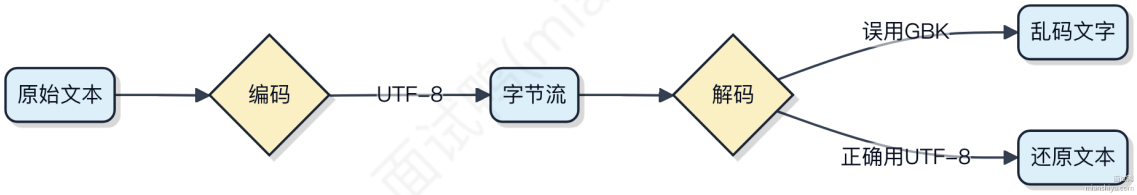
2） Web 应用里，前端页面声明的是 `UTF-8` ，但后端 Servlet 没设置 `request.setCharacterEncoding("UTF-8")` ，请求体里的中文就废了。

3） 数据库连接 URL 没加 `characterEncoding=utf8` ，存的时候就歪了。

代码上要盯住几个点：

```
// 读文件别用默认编码
new String(Files.readAllBytes(Paths.get("a.txt")), StandardCharsets.UTF_8);

// 写也要明确
Files.write(path, "内容".getBytes(StandardCharsets.UTF_8));
```

还有个坑是 `String.getBytes()` 和 `new String(bytes)` 这俩操作，不带参数就用平台默认编码，一换环境就崩。所以一定要显式传 `Charset` 。



只要记住：**所有涉及字节和字符转换的地方，都得明确指定字符集**，别偷懒。尤其是跨系统、跨语言、跨存储的时候，这个环节压根不经过大脑去猜，必须写死。

JDK 动态代理和 CGLIB 动态代理有什么区别？

JDK 动态代理和 CGLIB 的根本差异在于代理对象的生成方式和适用范围。

JDK 动态代理要求目标类必须实现至少一个接口，它通过 `java.lang.reflect.Proxy` 在运行时为接口创建代理实例。代理逻辑由 `InvocationHandler` 处理，方法调用会走到它的 `invoke` 方法里。这种方式不侵入原始类，但只能代理接口方法。

CGLIB 不依赖接口，而是通过继承目标类生成子类来实现代理。它使用 ASM 操作字节码，在子类中重写父类方法并插入拦截逻辑。这意味着 final 类或 final 方法无法被代理，因为不能被重写。

性能上，CGLIB 生成的代理类是具体子类，调用是直接的方法调用；而 JDK 代理多一层反射，早期版本慢一些，但从 Java 8 开始差距不大。现在选型更多看是否需要基于类代理。

Spring AOP 默认优先用 JDK 动态代理，只有当目标没有实现接口时才退化到 CGLIB。

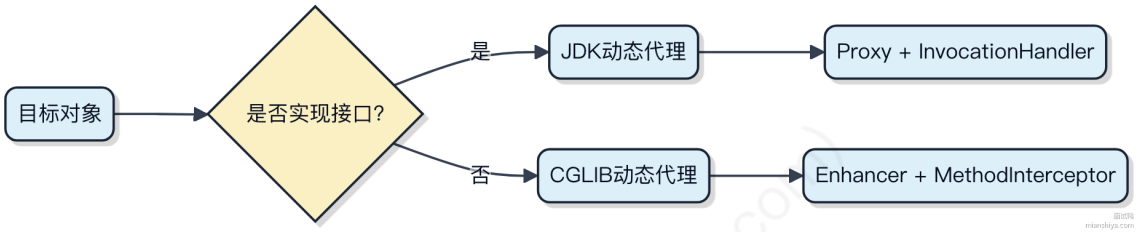
代码层面，JDK 代理的核心是：

```
Proxy.newProxyInstance(ClassLoader, interfaces, handler)
```

CGLIB 则是：

```
Enhancer.create(Class, Callback)
```

1）JDK 代理基于接口，CGLIB 基于继承
2）JDK 使用反射调度，CGLIB 是直接调用增强方法
3）CGLIB 能处理无接口的类，但无法代理 final 类或方法



Java 中的注解原理是什么？

注解本质是一个继承了 Annotation 接口的接口，你定义的每个注解在运行时都会生成一个动态代理实例，JVM 通过 AnnotationInvocationHandler 来管理属性和值。

1）编译时，javac 会处理源码中的注解，部分注解（如 @Override）会在这个阶段触发检查。如果使用了注解处理器（APT），像 Dagger 或 Lombok 就能在这个阶段生成新代码。

2）class 文件里会保留注解信息，靠的是 **Class 文件的 Attribute 结构**，比如 RuntimeVisibleAnnotations 和 RuntimeInvisibleAnnotations 这两个属性表，决定了注解是否保留到运行时。

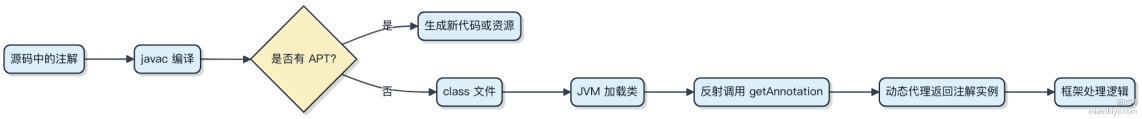
3）运行时通过反射获取注解，调用 Class、Method 或 Field 的 getAnnotation() 方法，底层其实是从 class 二进制数据中解析出注解数据，再通过动态代理还原成注解对象。

```
@Retention(RetentionPolicy.RUNTIME)
@interface MyConfig {
    String value();
}

public class Example {
    @MyConfig("test")
    public void run() {}
}
```

拿到方法上的注解：

```
Method m = Example.class.getMethod("run");
MyConfig ann = m.getAnnotation(MyConfig.class);
System.out.println(ann.value()); // 输出 test
```

注解本身不改变程序逻辑，它只是元数据。真正起作用的是读取这些注解的框架，比如 Spring 在启动时扫描 @Component，MyBatis 解析 @Select 注解绑定 SQL。



什么是 Java 的 SPI（Service Provider Interface）机制？

Java 里的 SPI 其实是一种**服务发现机制**，它让接口的实现类可以在运行时动态加载，而不是写死在代码里。

你定义一个接口，不同的厂商或模块提供各自的实现，JVM 在启动时会去读取 `META-INF/services/` 目录下的配置文件，自动把实现类加载进来。这个机制用得最典型的就是 JDBC。比如你写 `Connection conn = DriverManager.getConnection(url)` ，底层其实靠 SPI 把 MySQL、PostgreSQL 等驱动实现自动注册进来。

具体流程是这样的：
1）你在项目里引入 mysql-connector-java
2）这个 jar 包里有个文件叫 `META-INF/services/java.sql.Driver`
3）文件内容是 `com.mysql.cj.jdbc.Driver`
4）JVM 通过 `ServiceLoader` 读这个文件，反射加载并实例化这个类

```
ServiceLoader<Driver> loader = ServiceLoader.load(Driver.class);
for (Driver driver : loader) {
    System.out.println(driver);
}
```

这种设计的好处是解耦。框架只定义协议，具体实现由第三方提供，插件化扩展特别方便。Dubbo、Spring Boot 的自动装配也借鉴了这套思路。

不过要注意， `ServiceLoader` 是全量加载的，所有实现都会被创建出来，如果某个实现初始化很重，会影响性能。而且没有优先级控制，多个实现时顺序不确定。

Java 泛型的作用是什么？

泛型的本质是编译期的类型检查机制，它让集合类能记住装的是什么类型的对象。没有泛型的时候，往 ArrayList 里塞 String 或 Integer 都没问题，但取出来时容易 ClassCastException，得靠开发者自己记。

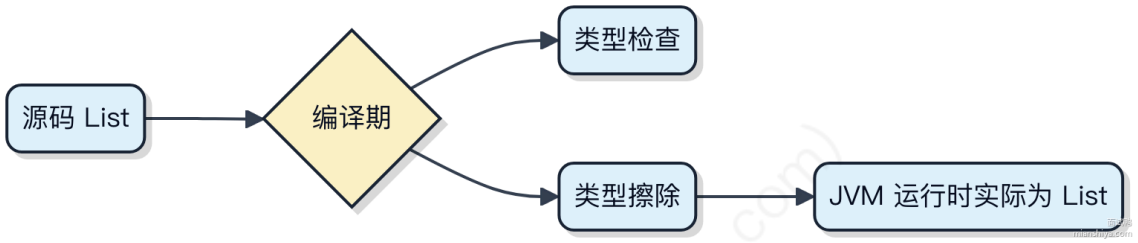
用了泛型之后，比如 `List<String>` ，编译器就会在编译阶段就报错，而不是运行时抛异常。这叫**类型安全**，其实底层字节码压根不带泛型信息，因为会经过**类型擦除**，也就是泛型只存在于源码和编译期，JVM 运行时根本看不到。

1）定义类或方法时用 `<T>` 表示类型参数，比如 `class Box<T> { T value; }`
2）调用时指定具体类型， `Box<Integer> box = new Box<>()` ，编译器自动做类型推断
3）不能用于静态变量，因为静态成员属于类，而泛型实例属于对象，生命周期不匹配

代码示例：

```
List<String> list = new ArrayList<>();
list.add("hello");
String s = list.get(0); // 不需要强转
```

常见坑是泛型数组不能直接创建，比如 `new List<String>[10]` 会编译失败。还有像 `List<Object>` 和 `List<String>` 没有继承关系，虽然 String 是 Object 子类，但泛型不继承类型关系。



什么是 Java 泛型的上下界限定符？

Java 泛型的上下界限定符，是用来约束类型参数取值范围的语法。它让你能更精确地控制泛型能接受哪些类型。

上界用 `extends` 关键字表示，意思是“这个类型必须是某个类或接口的子类（包括自身）”。比如 `List<? extends Number>` 就只能装 Integer、Double 这种 Number 的实现类，但不能装 String。这种场景在你只想从集合里读数据时特别有用，毕竟都是 Number，按父类处理没问题。

下界用 `super` 关键字，意思是“这个类型必须是某个类或其父类”。像 `List<? super Integer>` 可以存 Integer，也能存 Object。这种一般用在你要往集合写数据的场景，确保目标类型能接住你塞进去的东西。

有个经典例子就是 `Collections.copy()` 方法。它的逻辑是把一个列表的内容复制到另一个列表。源列表用 `? extends T`，只读；目标列表用 `? super T`，只写。这样既能保证类型安全，又能最大限度保持灵活性。

```
public static <T> void copy(List<? super T> dest, List<? extends T> src) {
    for (int i = 0; i < src.size(); i++) {
        dest.set(i, src.get(i));
    }
}
```

记住 PECS 原则：Producer-extends, Consumer-super。如果是生产数据的地方，用上界；消费数据的地方，用下界。

Java 中的深拷贝和浅拷贝有什么区别？

对象拷贝这事儿，关键看引用类型的成员变量怎么处理。

1）浅拷贝是把对象里的值都复制一份，基本类型没问题，但遇到数组、集合这些引用类型，只复制了引用地址。这就意味着，原对象和副本操作的是同一个堆内存里的数据，改一个，另一个也跟着变。

2）深拷贝会递归复制所有引用对象，直到每一层都是新创建的实例。两个对象彻底断开联系，互不影响。

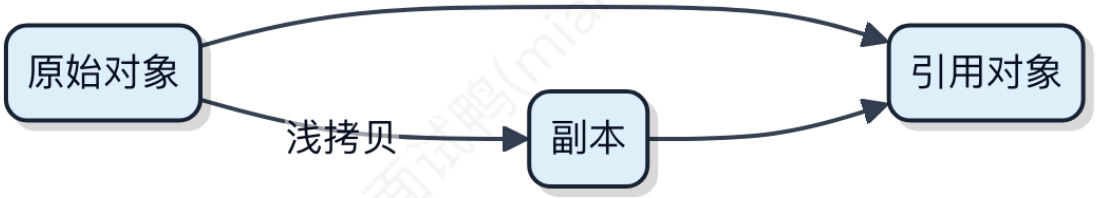
举个例子，比如你用 `Object.clone()`，默认就是浅拷贝。要想实现深拷贝，常见做法有几种。一种是重写 `clone()` 方法，在里面对引用类型手动 new 或 clone。另一种更省事的是序列化方案，比如用 Java 原生序列化或者 Kryo 这类库，直接把对象序列化再反序列化回来，相当于生成了一个全新的对象树。

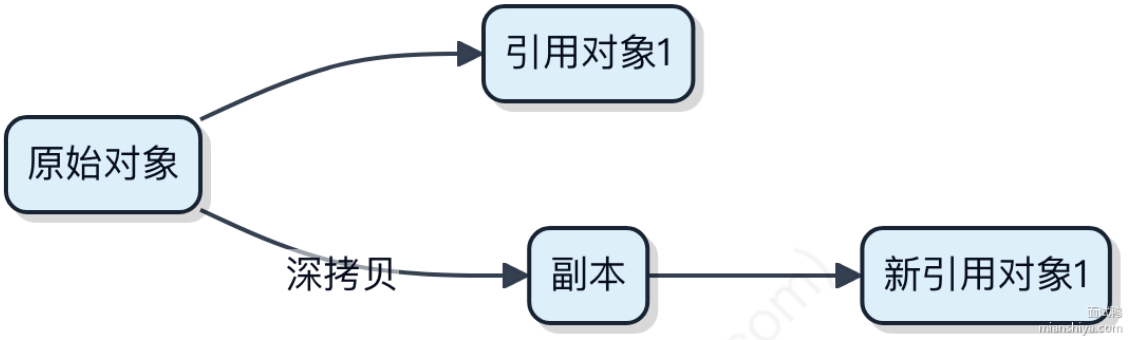
```
public class Person implements Cloneable {
    String name;
    Address address; // 引用类型

    public Person clone() throws CloneNotSupportedException {
        Person copy = (Person) super.clone();
```

```
        copy.address = (Address) address.clone(); // 手动深拷贝引用对象
        return copy;
    }
}
```

还有一种情况，如果引用对象是不可变的，比如 `String` 、 `Integer` ，那浅拷贝其实也够用，因为它们的值压根不能改。

要不要做深拷贝，得看业务场景。像配置对象、DTO 传输这种需要完全隔离的，就得深拷贝。要是只是临时读取，共享引用反而能节省内存。





什么是 Java 的 Integer 缓存池？

Java 里对 `Integer` 类型有个缓存机制，主要用来优化频繁使用的小整数对象的创建和内存消耗。这个缓存池在 JVM 启动时就初始化好了，范围默认是 -128 到 127。

1）当你用 `Integer.valueOf(100)` 或者自动装箱比如 `Integer i = 100` 时，如果值在这个范围内，JVM 不会新建对象，而是从缓存池里复用已有的实例。这能减少 GC 压力，提升性能。

2）但超出这个范围，比如 `Integer i = 200` ，就会走 new 路径，每次都是新对象。这也是为什么 `==` 比较在小数值上可能为 true，大数值却不行的根本原因。

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b); // true

Integer c = 200;
Integer d = 200;
System.out.println(c == d); // false
```

3）这个范围可以通过 `-XX:AutoBoxCacheMax=N` 参数调大，比如启动时加 `-XX:AutoBoxCacheMax=500` ，就能让 0~500 的值也被缓存。不过一般没必要改，除非你明确知道应用里有大量装箱操作集中在某个高位区间。

注意：不只是 `Integer` ， `Byte` 、 `Short` 、 `Long` 也有类似机制，范围固定在它们的最小到最大值之间（如 Long 也是 -128~127），而 `Float` 和 `Double` 没有缓存。

Java 的类加载过程是怎样的？

一个 Java 类从被加载到虚拟机内存中开始，直到卸载出内存为止，它的整个生命周期包括：加载、验证、准备、解析、初始化、使用和卸载七个阶段。我们重点关注前五个核心阶段。

1）加载 通过类的全限定名获取该类的二进制字节流，通常是从 class 文件、jar 包或者网络中读取。将字节流代表的静态存储结构转化为方法区的运行时数据结构，在堆中生成一个 `java.lang.Class` 对象作为入口。

2）验证 确保 Class 文件的字节流符合当前虚拟机的要求，不会危害虚拟机安全。比如格式检查、元数据检查、字节码检查等。这一步防止恶意代码搞破坏。

3）准备 为类变量（static 修饰的变量）分配内存并设置初始值。比如 `public static int value = 123;` 这里会先设成 0，而不是 123，真正的赋值要等到初始化阶段。

4）解析 将常量池内的符号引用替换为直接引用。比如某个方法调用了 `System.out.println()` ，这时候就会把符号引用定位到具体的方法内存地址上。

5）初始化 执行类构造器 `<clinit>()` 方法的过程，真正开始执行 Java 代码来初始化类变量。这个方法由编译器自动收集类中所有静态变量的赋值动作和静态代码块合并而成。

双亲委派模型就是在这个过程中起作用的机制，它保证像 `java.lang.Object` 这种核心类不会被自定义类加载器重复加载，避免安全问题。

什么是 Java 中的双亲委派模型？

类加载器在加载一个类时，不会自己先动手，而是把请求往上抛给父类加载器去尝试加载。这个过程一层层向上委托，直到最顶层的启动类加载器（Bootstrap ClassLoader），这就叫**双亲委派模型**。

比如你写了个 `java.lang.String` ，想搞点小动作。但系统类加载器收到请求后，会一路委托给 Bootstrap ClassLoader。它发现自己已经加载过核心包里的 String，直接返回，你的恶意类压根没机会加载。这样就保证了核心类库的安全性。

整个链条通常是这样的：应用类加载器 → 扩展类加载器 → 启动类加载器。每一级都优先让父级处理，只有父级搞不定时，才轮到子级出手。

当然也有例外，像 JDBC 或 JNDI 这种 SPI 场景就得打破双亲委派。这时候用 `Thread.getContextClassLoader()` 拿到当前线程的类加载器，反向向下委托，才能加载用户实现的驱动或服务。

自定义类加载器时，一般也不建议随便破坏这个模型，除非你真的清楚后果，不然容易引发类冲突或重复加载问题。

Java 中 hashCode 和 equals 方法是什么？它们与 == 操作符有什么区别？

Java 里每个对象都继承自 Object 类，所以天然就有 `hashCode` 和 `equals` 这两个方法。 `==` 操作符比较的是变量指向的内存地址，看是不是同一个实例。

`equals` 方法默认行为其实和 `==` 一样，也是比地址。但它的设计意图是用来判断**逻辑相等**。比如两个 User 对象，id 都是 1001，我们觉得它们是“同一个用户”，就得重写 `equals` 去比较关键字段。

`hashCode` 的作用是为对象生成一个整型哈希值，主要用在哈希结构里，比如 HashMap、HashSet。它有个硬性规定：如果两个对象 `equals` 返回 true，那它们的 `hashCode` 必须相同。反过来不成立，不同对象可以有相同哈希值，这就是哈希冲突。

这两个方法通常一起重写。你把对象放进 HashMap 当 key 时，流程是这样的：

```
// 举个例子
public class User {
    private Long id;

    @Override
    public boolean equals(Object o) {
        // 标准写法略去null和类型判断
        User u = (User) o;
        return id.equals(u.id);
    }

    @Override
    public int hashCode() {
        return id.hashCode();
    }
}
```

1）先调 `hashCode` 定位到桶位置
2）再用 `equals` 和桶里的每个元素比较，确认是否存在

要是只重写 `equals` 不重写 `hashCode` ，同一个逻辑对象可能被当成两个 key 存进去，HashMap 就乱了。这在实际开发中很容易踩坑，尤其是做缓存、去重的时候。

使用 new String("yupi") 语句在 Java 中会创建多少个对象？

这个问题看着简单，但得把字符串的底层机制理清楚。

首先，Java 的字符串有常量池这个概念。当代码里出现字面量比如 `"yupi"` 时，JVM 会在类加载阶段就把这个字符串放到**字符串常量池**里。所以 `"yupi"` 这个对象在常量池中只会有一份。

而 `new String("yupi")` 是运行时操作，它会强制在堆上创建一个**新的 String 对象**，哪怕常量池里已经有 `"yupi"` 了。这个新对象的内容是拷贝自常量池里的那个。

所以整个过程会创建几个对象？

1）如果 `"yupi"` 字符串是第一次使用，还没进常量池，那么先在常量池里创建一个。 2）然后 `new String()` 又在堆上创建一个，独立的对象。

也就是说，最多可能创建 **2 个对象**：1 个在常量池，1 个在堆。

但通常面试题默认 `"yupi"` 已存在于常量池（比如之前用过），那 `new String("yupi")` 就只会在堆上创建 1 个新对象。

关键点在于：**常量池对象是共享的，new 出来的对象是独立的，不走共享逻辑**。

```
// 示例
String s = new String("yupi");
// 如果 "yupi" 没出现过 → 2 个对象
// 如果 "yupi" 已存在 → 1 个对象（仅堆上）
```

判断到底几个，得看上下文。但标准答案一般是：**最多 2 个，最少 1 个**。

Java 中 final、finally 和 finalize 各有什么区别？

final、finally 和 finalize 这三个东西名字像，但完全是三码事，别搞混了。

final 是个关键字，用来修饰类、方法、变量。被它修饰的类不能被继承，比如 String 类就是 final 的。修饰方法时，这方法就不能被子类重写。修饰变量，那这个变量就成了常量，必须初始化，而且之后不能再改。基本类型值不变，引用类型的话，引用地址不能变，但对象内部数据还是可以改的。

```
final int x = 10;
// x = 20; // 编译报错
```

finally 是异常处理里的一个块，跟 try 搭配用。不管有没有异常，只要 JVM 没崩，finally 里的代码一定会执行。常用来做资源清理，比如关闭文件流、数据库连接。不过注意，如果在 try 里调用了 System.exit()，那就直接退出了，finally 也压根不经过。

```
try {
    // 可能出错的代码
} finally {
    // 总会执行，比如 close()
}
```

finalize 是 Object 类的一个方法，每个对象都有。以前设计是用来做垃圾回收前的清理工作，但**现在根本不推荐用**。因为它的执行时机完全不可控，可能永远不被调用，Java 9 开始已经标记为 deprecated 了。真要清理资源，应该用 try-with-resources 或者手动 close。

这三个词唯一共同点就是都以 "final" 开头，别的啥关系都没有。

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*